Adaptively Secure Blockchain-Aided Decentralized Storage Networks: Formalization and Generic Construction

Xiangyu Su[1⋆], Yuma Tamagawa[1], Mario Larangeira[1,2], and Keisuke Tanaka[1]

[1] Department of Mathematical and Computing Science, School of Computing, Institute of Science Tokyo. 2-12-1 Ookayama Meguro-ku, Tokyo, Japan. su.x.4029@m, tamagawa.y.f6c3@m, rebello.m.f72a@m, keisuke@comp.isct.ac.jp.
[2] Input Output, Singapore. mario.larangeira@iohk.io.

Abstract. A decentralized storage network (DSN) enables clients to delegate data to servers, where the data must be retained custody for a jointly agreed contract period. Most existing constructions employ proof-of-replication (PoRep) iteratively to provide verifiable guarantees during this period. However, the original PoRep security model does not account for iterative invocation, particularly when aided by a blockchain that serves as an external repository of auxiliary information. Similar concerns arise for another key DSN functionality, i.e., data retrieval, where malicious parties may exploit previous and parallel sessions to undermine fairness. This work addresses these gaps by formalizing the syntax and adaptive security of blockchain-aided DSN protocols, presenting, to the best of our knowledge, the first formal treatment. Building on this foundation, we propose a generic construction that leverages an Extended UTxO (EUTxO) ledger, automating protocol execution by embedding the required verification into EUTxO's validation logic. We further prove that the succinctness and non-malleability of the underlying proof system suffice for adaptive security. Our experiments corroborate this: systems lacking these properties are vulnerable to adaptive attacks, e.g., servers offloading storage while still producing valid PoRep proofs.

Keywords: Decentralized Storage Network · EUTxO-Based Blockchain · Adaptive Replication Security · Fair Data Retrieval

1 Introduction

Decentralization as a design strategy has long been used to avoid single points of failure in diverse settings. With the advent of blockchain and distributed ledger technologies, it has found further applications in anti-censorship mechanisms and public verification of actions. This blockchain-driven wave of novel systems was initially based on the proof-of-work (PoW) paradigm. Given its intense energy usage, several other approaches were proposed that differ in the

⋆ This work was supported by Input Output Global.

underlying resource. Whereas PoW consumes computing power, proof-of-stake (PoS) leverages the system's financial token. Another prominent direction is proof-of-space (PoSpace) [16], where the resource is *storage space*. This viewpoint naturally leads to related primitives such as proof-of-retrievability [30] and proof-of-replication (PoRep) [14, 19, 20].

Building on these primitives, a decentralized storage network (DSN), *e.g.*, [7, 28, 33, 40, 42, 44], enables clients to outsource data to untrusted servers while retaining publicly verifiable guarantees of correct behavior. In such systems, storage nodes maintain replicas of client data and periodically offer proofs (potentially via a blockchain) to demonstrate that the replicas remain intact. This advantage is amplified by replication across geographically distributed nodes, which mitigates risks such as natural disasters or localized failures. However, deploying DSN protocols in practice introduces several technical challenges, including verifying whether servers maintain the required replicas and enforcing fair data retrieval via secure payment and collateral mechanisms.

1.1 Technical Challenges

A DSN involves two types of participants: clients and servers. Each client delegates her data to one or more servers. Each server must preserve replicated forms of the data, *i.e.*, replicas, in its local storage for a mutually agreed contract period. At the end of that period[1], the client should be able to retrieve the data. Economic viability further requires the system to charge clients based on the size of their data, and to deter servers, via collateral, from using less storage than claimed or deleting the data. Accordingly, blockchain protocols are leveraged to aid DSN in two ways: (1) facilitating payments and collateral among participants; and (2) providing an immutable ledger to record evidence of data delegation, preservation, and retrieval. Concretely, we identify the following gaps in the current DSN literature.

A model for formal treatment. Perhaps surprisingly, most currently proposed DSN protocols, even those leveraging blockchain infrastructure, are based on the (Put, Get, Manage) abstraction [33]. Despite its applicability, this model is insufficient for an end-to-end formal security treatment. Follow-up studies, *e.g.*, [46], therefore analyze concrete attacks rather than well-defined properties.

Security during contract periods. The core building blocks intended to ensure that servers preserve replicas during the contract periods, *i.e.*, PoRep and its widely adopted variant, proofs of space-time (PoST) [36], do not account for iterative invocation, and thus lack an adaptive notion (see Section 1.2). More precisely, in a DSN with *stateful* servers that record responses to periodically-issued PoRep[2] challenges (as is typical in blockchain-aided protocols, where these

[1] On-demand retrieval is discussed as a configuration in Appendix C.1.

[2] For simplicity and consistency, we use the PoRep terminology throughout this paper, even though most observations apply equally to PoST.

entries are written on-chain), an adversary may compress the challenged replica block together with the on-chain response, thereby offloading its internal storage while still being able to answer subsequent challenges on the same block via rederivation. This behavior clearly violates the intended data preservation guarantee, yet does not contradict any existing PoRep security notion.

Although existing DSN constructions [33, 46] employ zero-knowledge (ZK) protocols (for privacy) or even zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs) (for privacy and efficiency) to mask on-chain responses, they often overlook the *security* implications: ZK alone does not prevent the aforementioned breach. Our experiments in Section 5 target precisely this scenario, *i.e.*, embedding a PoRep without ZK or succinctness into a DSN, to demonstrate that both properties are also essential for security.

When we zoom in further on the proof aspect of PoRep, the adaptive security gap exposes DSN to another well-studied threat from the proof-system literature: proof malleability, which is particularly problematic in a distributed setting. In particular, the malleability of proofs recorded on-chain may allow an adversary to derive new valid proofs without fully storing the corresponding replica blocks.

Security for data retrieval. On the other hand, secure data retrieval requires that: the recovery of the remaining collateral and delivery of the data must be performed *atomically*. The corresponding property, *i.e.*, fairness for both parties [41], is formalized only in the context of data exchange. However, since a DSN stores the same piece of data under different replicas, a direct adoption of these fairness notions may suffer from the same adaptive security concern arising from proof malleability.

Leveraging the ledger for payment/collateral management. The UTxO Model has global reach, underpinning widely used currency tokens, *e.g.*, Bitcoin and Cardano. Its extended variant, *i.e.*, the Extended UTxO (EUTxO) model [11], augments UTxO with the ability to express arbitrary validation logic while preserving *deterministic* and locally verifiable state transitions, unlike the inherently non-deterministic transitions found in the Account Model [45]. This determinism enables a much tighter integration of transactions into our proposed DSN protocol design, leading to more expressive, adaptively enhanced security definitions that pave the way for safe payment and collateral management. To the best of our knowledge, such an EUTxO-driven approach to DSN has not been explored.

1.2 Related Works

Our literature review focuses on two aspects: PoRep and fair data retrieval. We do not cover rational analysis, *e.g.*, [12], as it is orthogonal to our focus.

The replication soundness of PoRep, formalized in [19,20] under the Random-Access Machine (RAM) Model, ensures that if a prover can produce multiple valid PoRep proofs within a restricted time frame, it must be storing a corresponding number of independent data replicas. A definition without timing assumptions, which is more closely aligned with standard knowledge soundness,

is later provided in [14]. These formulations are not designed for iterative invocation, and the vanilla scheme achieves replication soundness without even mandating ZK or succinctness[3] of its proofs.

A space-time variant, PoST [36], is proposed as a public-coin PoRep that supports multiple rounds of challenge–response. However, its soundness [36, Definition 3, 4] does not capture the potential adaptivity across execution rounds. To the best of our knowledge, the closest prior work that considers adaptivity appears in [2], which formalizes *continuous extractability* in the Interactive Turing Machine (ITM) Model. That notion, however, focuses on extractability rather than quantifying space usage (as in PoSpace or PoRep), consistent with the fact that the scheme is a proof of *storage-time* rather than *space-time*.

On the other hand, the requirements of data retrieval are conceptually close to those of atomic swap [15] and data exchange [41]. Theoretically, client-fairness and server-fairness are formalized within the game-based paradigm to capture security against malicious servers and clients, respectively [41]. In practice, most existing constructions are built atop Ethereum-type smart contracts [45], yet the same functionality can also be realized in the UTxO Model using adaptor signatures, a primitive originally proposed as scriptless scripts [38] (and later formalized in [4]) for conditional payments on the Bitcoin platform. However, as noted in [13, 25], the original security notions for adaptor signatures in [4] are insufficient for sophisticated applications. This motivates us to refine the fairness formalization for adaptor signature-based data retrieval and, given our observation in Section 1.1, to further strengthen it *w.r.t.* adaptive security.

1.3 Our Contributions

At a high level, our contribution is a thoroughly described adaptively secure DSN protocol aided by an EUTxO-based blockchain. Realizing this requires overcoming several technical challenges and addressing the current limitations of the state of the art.

Ledger leverage for DSN model. This work introduces 4-phase protocol, conceptually aligned with the $(\mathsf{Put}, \mathsf{Get}, \mathsf{Manage})$ abstraction: (1) preparation, (2) data delegation, (3) data preservation via automatic test loops, and (4) data retrieval. The protocol allows a client to hand data to servers, which store replicas and provide proofs throughout the contract period automatically. Crucially, payment, collateral, and all proofs are handled entirely by a carefully specified EUTxO execution logic, without any client intervention, thereby achieving public verifiability. To the best of our knowledge, this is the first end-to-end formalization of a blockchain-aided DSN protocol, instantiated with EUTxO transactions and achieving publicly verifiable preservation and retrieval. The resulting protocol is also highly flexible, with configurations presented in Appendix C.1.

[3] zkSNARKs are mentioned in [20] solely for efficiency.

Full DSN security description. We propose a comprehensive set of security definitions for our blockchain-aided DSN model, addressing the gaps we identified in the existing PoRep, PoST, and DSN literature.

Notably for replication soundness, we start from the original notion [14, 20] and lift it to the adaptive setting by using standard adaptive security techniques, with tweaks tailored to PoRep: the explicit challenge–response matrix (arising from PoRep's inherent parallelism and iterative invocation) is replaced by oracle access. This avoids brittle index alignments and yields our adaptive replication soundness notion. We further prove that, in the blockchain setting (under the knowledge of strong compression (KoSC) assumption [21]), adaptive replication soundness for a DSN requires compiling the underlying PoRep with a proof system that provides ZK, succinctness, and simulation-extractability[4].

As for fairness, the client- and server-side notions of [41] are first strengthened using the up-to-date adaptor signature security model of [13]. For client-fairness, we further incorporate a simulation-extractability-style enhancement to address adaptive attacks arising from proof malleability. In contrast, CCA2-style adaptivity is not applied to server-fairness, as both existing protocols and ours handle it via a one-time encryption mechanism.

Generic construction and experimental motivation. We provide a provably secure (under our formalization) generic construction that serves as a blueprint for blockchain-aided DSN protocols, with each component expressed via modular building blocks. Our design choices are supported by experimental evidence: we show that an adversarial server can tamper with the randomness in a non-succinct PoRep instantiated within a DSN to mount an adaptive attack, either (1) by embedding information into the randomness via a subliminal channel; or (2) by choosing a compressible random value and storing it in place of an entire replica fragment. We demonstrate both attacks using toy examples in Section 5. Although we did not conduct system-level benchmarks, we provide a brief scalability and cost evaluation based on real-life figures in Appendix D.

1.4 Organization

The basic notation and cryptographic primitives used as building blocks later are reviewed in Section 2, including an EUTxO Model ledger and PoRep (more basic definitions in Appendix A). Next, Section 3 offers an overview of the protocol, including the description of the blockchain-aided DSN model, the introduction of the main subprotocols, namely the *initial setup*, *data delegation*, *data preservation*, and *data retrieval* phases, with their syntax and the security intuition. Section 4 complements the earlier section by presenting a concrete construction of the outlined subprotocols. Section 5 fully motivates our design choices and security definitions. Finally, we conclude our work in Section 6.

[4] Hence, a simulation-extractable zkSNARK is sufficient to build an adaptively replication secure blockchain-aided DSN; and fortunately, most widely-used universal zkSNARKs already satisfy or can efficiently achieve simulation-extractability [17].

2 Preliminaries

We use λ for the security parameter; $\mathsf{poly}(\cdot)$ and $\mathsf{negl}(\cdot)$ denote a polynomial and a negligible function, respectively; and PPT is short for probabilistic polynomial time. For any integers $a < b$, let $[a \,..\, b] \coloneqq \{a, a+1 \ldots, b\}$. For any integer $n > 0$ (*resp.*, $n \geq 0$), let $[n] \coloneqq [1 \,..\, n]$ (*resp.*, $[n]_0 \coloneqq [0 \,..\, n]$). For a set (or list) X, $|X|$ denotes the size of X, and $x \xleftarrow{\$} X$ denotes that x is randomly and uniformly sampled from X. For an algorithm Alg, $x \xleftarrow{\$} \mathsf{Alg}$ denotes that x is assigned the output of Alg on fresh randomness.

Next, we present the definitions of our necessary building blocks, including an Extended UTxO (EUTxO)-based blockchain protocol and a proof-of-replication (PoRep) scheme. Additional standard components, such as symmetric encryption SKE, signature schemes SIG, relation primitives R for NP relations[5], and non-interactive argument protocols are provided in Appendix A.1; and the syntax and security of adaptor signatures are provided in Appendix A.2.

2.1 EUTxO-Based Blockchain Protocol

This work assumes a secure EUTxO-based blockchain protocol. Here, we recall the security of blockchain protocols, *i.e.*, persistence and liveness, from [23][6].

Definition 1 (Persistence and Liveness). *For any non-negative $k, u \in \mathbb{N}$, a secure blockchain protocol proceeds in rounds with a ledger $\mathcal{L}$, denoted by $\Pi_{\mathcal{L},k,u}$, satisfies the following properties:*

- *k-persistence: In any round, if an honest user reports $\mathcal{L}$ that contains a message m in a block at least k blocks away from the end of $\mathcal{L}$, then m is reported by all honest users in the same position on $\mathcal{L}$ from that round onward;*
- *(k, u)-liveness: If a message m is given as input to all honest users continuously for u consecutive rounds, then all honest users report this message at least k blocks from the end of $\mathcal{L}$.*

As for the EUTxO Model [11], it extends the UTxO Model [3] by introducing a more expressive form of validation scripts. This enhancement enables the implementation of general state machines while preserving semantic simplicity, *i.e.*, the validation of state transitions is solely determined by output-input pairs in connected transactions (which form a directed acyclic graph), eliminating the need to maintain a historical global state as in the Account Model [45].

Let $\Pi_{\mathcal{L},k,u}$ be a secure EUTxO-based blockchain protocol. A transaction tx, *submitted* to $\Pi_{\mathcal{L},k,u}$, consists of an output list O, an input list I, and an optional

[5] Combining SIG and R yields an adaptor signature scheme, as described in [13].

[6] More precisely, these properties resemble the requirements for a "robust public transaction ledger" in *loc. cit.*, which can be implemented, in a bounded-synchronous setting, by a blockchain protocol that satisfies the more widely adopted common prefix, chain growth and chain quality properties. Hence, we use "ledger" and "chain" as interchangeable terms throughout this work.

validity interval vi (used to implement time-locks), *i.e.*, denote $\mathsf{tx} \coloneqq (O, I, vi)$. Consider another transaction tx' *s.t.* an input of tx spends an output of tx'. In this context, tx' is referred to as a *parent* transaction of tx, and tx as a *subsequent* transaction of tx'. We describe the data structure of EUTxO transactions *w.r.t.* tx and tx' in a simplified form based on [11, Figure 3][7].

- For each $i \in [|\mathsf{tx}'.O|]$, the output is denoted by $\mathsf{tx}'.\mathsf{out}_i \coloneqq (\nu, value, \delta)$ where:
  - ν is the validator *script* that determines whether an input can spent $\mathsf{tx}'.\mathsf{out}_i$;
  - *value* specifies the amount of cryptocurrency (used in $\Pi_{\mathcal{L},k,u}$) that is *locked* in $\mathsf{tx}'.\mathsf{out}_i$, *i.e.*, available to be spent by an eligible input;
  - δ is the datum that contains arbitrary contract-specific information.
- For each $j \in [|\mathsf{tx}.I|]$, the input is denoted by $\mathsf{tx}.\mathsf{in}_j \coloneqq (\mathsf{tx}'.\mathsf{out}_i, \rho)$ where:
  - $\mathsf{tx}'.\mathsf{out}_i$ is (the reference to)[8] the i-th output of tx', which is spent by $\mathsf{tx}.\mathsf{in}_j$;
  - ρ is the redeemer that proves the eligibility of $\mathsf{tx}.\mathsf{in}_j$ to spend $\mathsf{tx}'.\mathsf{out}_i$ *w.r.t.* the validator $\mathsf{tx}'.\mathsf{out}_i.\nu$. Particularly, if ρ requires a signature σ, we denote the base redeemer (*i.e.*, unsigned) $\tilde{\rho} \coloneqq \rho \backslash \{\sigma\}$.
- vi is the interval of rounds (cf. Definition 1), during which tx can be processed.

For convenience, we extend the tilde notation to transactions,*i.e.*, given tx as above, its *base transaction*, with all signatures unattached, is denoted by $\tilde{\mathsf{tx}} \coloneqq \mathsf{tx} \backslash \{\sigma_j\}_{j \in [|\mathsf{tx}.I|]}$, where $\sigma_j \in \rho_j$ is the signature in the j-th input redeemer.

Next, the validation of tx (cf. [11, Figure 6]) is divided into two subprocesses:

1. Checking whether tx is *well-formed*, including the following verifications: the current round of $\Pi_{\mathcal{L},k,u}$ is within the validity interval of tx; all outputs have non-negative values; all inputs refer to unspent outputs; the outputs preserve the unspent value recorded in $\Pi_{\mathcal{L},k,u}$ (a fundamental requirement inherited from the UTxO model); and no input double spends existing outputs;
2. Verifying the *eligibility* of each input of tx to spend its corresponding output. Consider $\mathsf{tx}'.\mathsf{out}_j$ and $\mathsf{tx}.\mathsf{in}_i$ as an example, and parse $\mathsf{tx}'.\mathsf{out}_i = (\nu_i, value_i, \delta_i)$. We say $\mathsf{tx}.\mathsf{in}_j$ is eligible to spend $\mathsf{tx}'.\mathsf{out}_i$ if, and only if,

$$\nu_i(\delta_i, \mathsf{tx}.\mathsf{in}_j.\rho, \mathsf{tx}) = 1. \tag{1}$$

Note that ν_i takes as input the entirety of tx, which was formally given as the context type in [11]. This is to enforce contract continuity, *i.e.*, the same contract code is used throughout the sequence of transactions. For brevity, we may write $\nu_i(\mathsf{tx}.\mathsf{in}_j.\rho) = 1$ when the context is clear.

Since this work focuses on state transitions triggered by transactions that reflect underlying protocol logic, we abstract the first subprocess with an oracle $\mathcal{O}_{\mathsf{txVrfy}}$ *s.t.* tx (*resp.*, its base transaction $\tilde{\mathsf{tx}}$) is said to be *well-formed* if, and only

[7] Particularly, in *loc. cit.*, the validator and datum are relocated to transaction inputs, with only their hash values retained as references in the outputs to improve memory efficiency. However, we revert this modification and treat the validator and the datum as components of the outputs for conceptual simplicity.

[8] In [11], the reference is specified as a pair of tx's identifier (*e.g.*, its hash) and the index i. We omit all such references to reduce notations.

if, $\mathcal{O}_{\mathsf{txVrfy}}(\mathsf{tx}) = 1$ (*resp.*, $\mathcal{O}_{\mathsf{txVrfy}}(\tilde{\mathsf{tx}}) = 1$). We incorporate this oracle into $\Pi_{\mathcal{L},k,u}$ and denote the resulting protocol by $\Pi^{\mathcal{O}_{\mathsf{txVrfy}}}_{\mathcal{L},k,u}$.

For the second subprocess, when referring to the construction of a concrete validator ν, we may explicitly write it as an algorithmic script of the form:

$$\nu := \textsc{Script}(\rho, \mathsf{tx})\{\text{validation scripts}\}.$$

For example, let ρ be a signature of a transaction tx under (sk, pk), *i.e.*, $\rho = \sigma \leftarrow \mathsf{SIG.Sign}(sk, \mathsf{tx}\backslash\{\rho\})$, we denote the validator that verifies the pair (ρ, tx) as

$$\nu = \textsc{Script}(\rho, \mathsf{tx})\{\mathsf{SIG.Vrfy}(pk, \mathsf{tx}\backslash\{\rho\}, \rho)\}.$$

Finally, we say a transaction tx is *confirmed* on the ledger $\mathcal{L}^{k,u}$ of $\Pi^{\mathcal{O}_{\mathsf{txVrfy}}}_{\mathcal{L},k,u}$, denoted by $\mathsf{tx} \in \mathcal{L}^{k,u}$, if and only if: (1) $\mathcal{O}_{\mathsf{txVrfy}}(\mathsf{tx}) = 1$; (2) for every input of tx and its corresponding output, Eq. 1 holds; and (3) tx is included in a block that is $k' \geq k$ blocks away from the end of $\mathcal{L}^{k,u}$ (implying a *maximum* of $k' + u$ rounds since tx was *submitted*). For simplicity, we may omit parameters k, u when describing confirmed transactions, *e.g.*, $\mathsf{tx} \in \mathcal{L}$.

2.2 PoRep Scheme

We formally present the PoRep [19,20] scheme and describe PoST [36], a public-coin PoRep with periodically issued challenges, which is a key building block in existing DSN protocols [33, 46].

Syntax. A PoRep scheme, denoted PoRep, consists of a tuple of algorithms $(\mathsf{Setup}, \mathsf{Preproc}, \mathsf{Replicate}, \mathsf{Extract}, \mathsf{Poll}, \mathsf{Prove}, \mathsf{Vrfy})$. All algorithms are assumed to operate in the Random-Access Machine (RAM) Model of computation, and parallel algorithms operate in the Parallel RAM (PRAM) Model.

- $\mathsf{Setup}(1^\lambda, t)$ is a one-time setup that takes as input the security parameter λ and a time parameter t that determines the length of a challenge-response period. It outputs the public parameters pp;
- $\mathsf{Preproc}(\mathsf{pp}, \tilde{D})$ is a preprocessing algorithm that takes as input pp and the data $\tilde{D} \in \{0,1\}^*$ with size up to $O(\mathsf{poly}(\lambda))$, with a potentially empty (*i.e.*, $\perp$) encoding key (see Remark 2). It outputs the preprocessed data D and a corresponding data tag τ_D;
- $\mathsf{TagVrfy}(\mathsf{pp}, D, \tau_D)$ is a deterministic algorithm that takes as input pp and a data-tag pair (D, τ_D). It outputs 1 if D is consistent with τ_D, or 0 otherwise;
- $\mathsf{Replicate}(\mathsf{pp}, id, D, \tau_D)$ takes as input pp, a replication identifier id, preprocessed data D and its corresponding data tag τ_D. It outputs a replica R and a piece of (compact) auxiliary information aux;
- $\mathsf{Extract}(\mathsf{pp}, id, \tau_D, \mathsf{aux}, R)$ takes as input pp, a replication identifier id, a data tag τ_D, auxiliary information aux, and a replica R. It outputs D extracted from R if (id, D, R) is consistent with (τ_D, aux), or $\perp$ otherwise.

The remaining $(\mathsf{Poll}, \mathsf{Prove}, \mathsf{Vrfy})$ composes an interactive challenge-response protocol executed between a prover and a verifier.

- $\mathsf{Poll}(\mathsf{pp}, \mathsf{aux})$ takes as input pp and auxiliary aux. It outputs a public challenge ch and a potentially empty (*i.e.*, $\perp$) secret randomness r;
- $\mathsf{Prove}(\mathsf{pp}, id, R, \mathsf{aux}, \mathsf{ch})$ takes as input pp, a replication identifier id, a replica R, auxiliary aux, and a challenge ch. It outputs a proof π *w.r.t.* ch;
- $\mathsf{Vrfy}(\mathsf{pp}, id, \tau_D, \mathsf{aux}, \mathsf{ch}, r, \pi)$ takes as input pp, a replication identifier id, a data tag τ_D, auxiliary information aux, a public challenge ch alongside its randomness r, and proof π. It outputs 1 if the proof is valid, or 0 otherwise.

In certain specifications [19], which we adopt as a building block in Section 4, aux is explicitly denoted by $(\tau_R, \mathsf{ch}_{\mathsf{aux}}, \pi_{\mathsf{aux}})$, where τ_R is a replica tag, and $(\mathsf{ch}_{\mathsf{aux}}, \pi_{\mathsf{aux}})$ is the initial challenge-proof pair generated during $\mathsf{Replicate}$. To accommodate this case, we may overload the replication interface as $(R, \tau_R, \pi_{\mathsf{aux}}) \leftarrow \mathsf{Replicate}(\mathsf{pp}, id, D, \tau_D, \mathsf{ch}_{\mathsf{aux}})$ and the initial verification as $\mathsf{Vrfy}(\mathsf{pp}, id, \tau_D, \mathsf{aux})$.

Remark 1 (Public-coin PoRep and PoST [36]). A PoRep scheme is called public-coin if ch can be derived from a publicly verifiable source, in which case $r = \perp$. This property enables the construction of a PoST scheme [36], in which the challenge-response protocol can be non-interactively chained by using the last response as a seed, denoted by seed, to generate the next challenge. Our DSN design also employs this space-time variant to ensure that a server faithfully stores the delegated data throughout the contract period (Section 4.2). Accordingly, we treat Poll as *deterministic w.r.t.* a fixed public seed, *i.e.*, $\mathsf{ch} \leftarrow \mathsf{Poll}(\mathsf{pp}, \mathsf{aux}, \mathsf{seed})$, and omit the randomness r from Vrfy, *i.e.*, $\mathsf{Vrfy}(\mathsf{pp}, id, \tau_D, \mathsf{aux}, \mathsf{ch}, \pi)$.

Security. A PoRep scheme should satisfy correctness and *rational* replication soundness. While overall description follows [19], the definition of rational replication soundness is refined to more closely align with the extractor-based formulation of [14][9].

Definition 2 (Correctness). *A PoRep scheme is correct if, for any $\lambda > 0$, id and preprocessed data D, there exists a t_λ s.t. the following probabilities hold, for all $t \geq t_\lambda$, $\mathsf{pp} \leftarrow \mathsf{Setup}(\lambda, t)$ and $(R, \mathsf{aux}) \leftarrow \mathsf{Replicate}(\mathsf{pp}, id, D, \tau_D)$,*

$$\Pr\left[\mathsf{Vrfy}(\mathsf{pp}, id, \tau_D, \mathsf{aux}, \mathsf{ch}, \pi) = 1 \,\middle|\, \begin{array}{c} \mathsf{ch} \leftarrow \mathsf{Poll}(\mathsf{pp}, \mathsf{aux}); \\ \pi \leftarrow \mathsf{Prove}(\mathsf{pp}, id, R, \mathsf{aux}, \mathsf{ch}) \end{array}\right] = 1,$$

s.t. $\mathsf{Prove}(\mathsf{pp}, id, R, \mathsf{aux}, \mathsf{ch})$ runs in parallel time at most t_λ; and D is guaranteed to be retrieved from R. Particularly, we say that R is retrievable:

$$\Pr\left[D' = D \,|\, D' \leftarrow \mathsf{Extract}(\mathsf{pp}, id, \tau_D, R)\right] = 1.$$

At a high level, an ideal replication soundness requires that any prover who simultaneously passes verification in $k \geq 1$ distinct PoRep protocols, each under

[9] Details on the original replication soundness of [19] is provided in Appendix B.1.

a unique identity but potentially with identical preprocessed data as input, must be storing k independent replicas.

Consider an experiment for an adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ (illustrated in Figure 1). $\mathcal{A}_1$ is allowed to select data inputs (*i.e.*, data of length N and their corresponding valid data tags) and the number of replicas $k \geq 1$. It also choose k distinct identifiers for each data-tag pair. The validity of a data-tag pair is given by a tag-verification oracle, denoted by $\mathcal{O}_{\mathsf{Check}}(D, \tau_D)$. $\mathcal{A}_1$ outputs an advice string η (which may be considered as the concatenation of k replicated data), alongside the replication auxiliary $\vec{\mathsf{aux}}$. Upon receiving a vector of k challenges from the challenger, $\mathcal{A}_2$ must output k proofs using $(\eta, \vec{\mathsf{aux}})$ from $\mathcal{A}_1$ (*i.e.*, without the data $\vec{D}$). The challenger then runs Vrfy on each proof for $i \in [k]$. Ideally, the experiment should output the conjunction of all verification results, and the adversary is deemed successful if the experiment returns 1.

However, enforcing the adversary to store all k replicas in an retrievable format has been proven impossible under purely cryptographic assumptions [19, Proposition 1]. Instead, a rational variant is considered in *loc. cit.*. Specifically, the rational replication soundness (referred to simply as replication soundness throughout the paper) asserts that: if an adversary can successfully answer v challenges (*i.e.*, if Vrfy outputs 1) by storing an advice string η, then there must exist an extractor $\mathcal{E}$ that can successfully output u retrievable replicas *s.t.* the probability of either $u < v$ or $|\eta| < (1-\epsilon)v|R|$ is negligible in λ, where $|R|$ denotes the length of an retrievable replica.

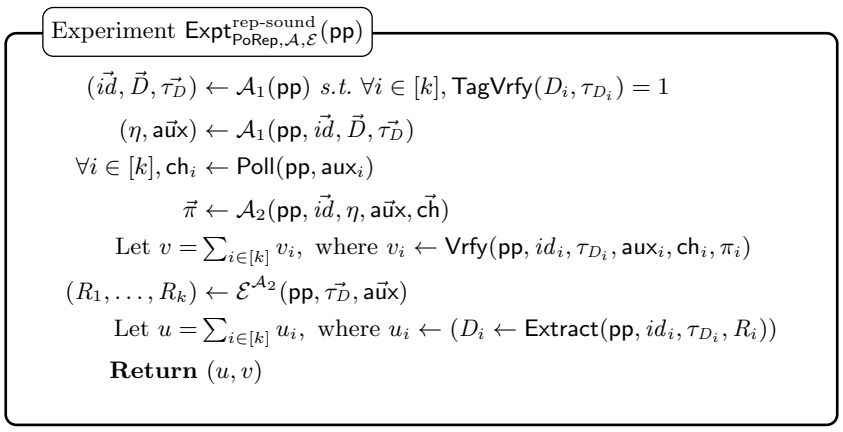


Fig. 1: The experiment for replication soundness. The adversary $\mathcal{A}$ receives pp. The experiment ends by returning the number of successfully answered challenges from $\mathcal{A}$ and that of successfully extracted retrievable replica from extractor $\mathcal{E}^{\mathcal{A}_2}$, respectively.

Definition 3 (Replication Soundness). *A PoRep is ϵ-replication sound if, for any adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, where $\mathcal{A}_1$ runs in $O(\mathsf{poly}(\lambda))$ and $\mathcal{A}_2$ runs in parallel time at most t, there exists an extractor $\mathcal{E}^{\mathcal{A}_2}$ s.t. the following*

$$\Pr\left[u < v \vee (|\eta| < (1-\epsilon)v|R|) \middle| (u, v) \leftarrow \mathsf{Expt}^{\text{rep-sound}}_{\mathsf{PoRep},\mathcal{A},\mathcal{E}}(\mathsf{pp})\right] \tag{2}$$

is $\mathsf{negl}(\lambda)$ *for any* $\lambda, t > 0$ *and* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, t)$.

In particular, this work adopts the generic PoRep construction with μ soundness error (Eq. 2) from [19, Lemma 2]. The KoSC assumption is used implicitly, as it underpins the replication soundness of the adopted construction. Moreover, a (μ, ϵ)-sound PoRep yields a $((1-\epsilon)kN, t, \mu)$-sound PoSpace [19, Lemma 1] (where $N = |D|$) under the time bounded incompressibility notion defined in *loc. cit.*. The formal definitions, assumptions, and lemmas referenced are collected in Appendix B.2 for completeness.

3 Our Blockchain-Aided DSN Formalization

This section presents our formalization of an EUTxO-based blockchain-aided DSN protocol in terms of syntax and security definitions. We begin with an overview of the protocol.

3.1 Overview: Phases and Security Properties

In our blockchain-aided DSN protocol, each client holds an independent input data $\tilde{D}$. Our formal description focuses on a single client entity $\mathcal{C}$, which may consist of multiple clients who do not necessarily trust each other. The data to be delegated, denoted by D, is generated in a preprocessing algorithm (detailed in Remark 2). With the preprocessed data D, the client entity $\mathcal{C}$ delegates it to one or more servers. For simplicity, we consider a single server $\mathcal{S}$ scenario, while the multi-server case can be realized by assigning distinct contract identifiers that embed server information to each data delegation. These identifiers are incorporated into our formal syntax in Section 3.2.

Protocol phases. After data preprocessing, the remainder of our protocol is divided into three phases.

- *Delegation phase*: $\mathcal{C}$ delegates the data D to $\mathcal{S}$ off-chain. To establish on-chain evidence, they jointly issue a contract transaction tx_{con} that embeds the contract metadata (*i.e.*, identifier, period, and transaction validity intervals) and delegation outcomes (*i.e.*, data tag and replication auxiliary information). It also locks payment from $\mathcal{C}$ and collateral from $\mathcal{S}$ in the ledger;
- *Preservation phase*: Throughout the contract period, $\mathcal{S}$ periodically submits test transactions tx_{test} in a timely manner, *without interaction from* $\mathcal{C}$. Each transaction embeds a publicly verifiable proof guaranteeing the storage of replicas of the delegated data D;
- *Retrieval phase*: By the end of the contract period[10], $\mathcal{C}$ is handed back the data D from $\mathcal{S}$, facilitated by a finalization transaction tx_{fin} published in the ledger. The transaction concludes the protocol and distributes the payment and collateral based on the retrieval outcome.

[10] A client may seek early data retrieval, either as part of terminating the contract or as an on-demand request. A detailed discussion, including how deployed DSN protocols handle data retrieval, is deferred to the configuration section in Appendix C.1.

Security properties. Here, we briefly describe the security properties of our protocol. Note that security *within* a client entity consisting of multiple clients is out of the scope of this work. The formal definitions are presented in Section 3.3.

- *Correctness* means that, for each phase in the protocol, both the off-chain outputs and on-chain transactions are verifiable if parties follow the prescribed rules defined by the protocol algorithms;
- *Delegation binding and contract unforgeability* guarantee that no adversary can unilaterally produce a valid contract transaction corresponding to an invalid replica (*i.e.*, one cannot be extracted to the delegated data) or embedding invalid off-chain delegation outcomes;
- *Adaptive replication soundness* constitutes one of our main contributions. It extends the replication soundness [19] (Definition 3) to an adaptive setting, ensuring that no adversary can break this security requirement throughout the entire contract period, even if it can access prior proofs recorded on the blockchain via test transactions and adaptively manipulate its local storage;
- *Adaptive client-fairness* extends the client-fairness notion from [41], which ensures that any *honest client* either successfully retrieves her data and produces a valid finalization transaction that refunds the server's collateral; or the collateral is forfeited. Our extension considers a more realistic setting, where an adversarial server has access to transcripts from other retrieval sessions on potentially identical data, and may thereby deceive the client into producing a valid tx_{fin} even when no data can be retrieved;
- *Server-fairness* is defined analogously to that in [41], which requires that any *honest server* either has her collateral refunded via a valid finalization transaction, with the client successfully retrieving her data; or the client learns no information about the data. Unlike client-fairness, we eliminate potential adaptive attacks by employing a one-time-use key for each retrieval session, similar to the ephemeral key model in [26].

3.2 Syntax of Protocol Phases

Let $\Pi_{\mathcal{L},k,u}^{\mathcal{O}_{\mathsf{txVrfy}}}$ ($\Pi_{\mathcal{L}}$ for short) be an EUTxO-based blockchain, where each party is identified by a key pair from a signature scheme $\mathsf{SIG} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ (Appendix A.1), *i.e.*, $(sk, pk) \leftarrow \mathsf{SIG.KGen}(1^\lambda)$. This section presents the syntax of a DSN protocol built atop $\Pi_{\mathcal{L}}$, denoted by $\mathsf{DSN}_{\mathcal{L}}$. Certain design aspects, including incentive models, scalability optimizations, and retrieval policies, are configurable and discussed separately in Appendix C.1.

Preprocessing phase. We begin with the setup and preprocessing algorithms.

- $\mathsf{Setup}(1^\lambda, t)$ is a one-time procedure performed for each DSN instance, either externally or cooperatively by the involved parties. It takes as input the security parameter λ and a time parameter t. Setup outputs public parameters pp *s.t.* $(\mathsf{crs}, pk_{\mathcal{B}}, T(\lambda, t)) \in \mathsf{pp}$, where crs is a common reference string; $pk_{\mathcal{B}}$ is a designated burning address in $\Pi_{\mathcal{L}}$ that forfeits the collateral upon a detected

protocol violation; and $T(\lambda, t)$ (T for short) specifies the length of the testing interval, measured in rounds of $\Pi_{\mathcal{L}}$, during the Preservation phase. As pp is implicitly taken by all algorithms below, we omit it when the context is clear;
- $\mathsf{TSetup}(1^\lambda, t)$ is like Setup, *i.e.*, it outputs pp and additionally a trapdoor td;
- $\mathsf{Preproc}(ek, \tilde{D})$, run by a client entity, takes as input a *secret* encoding key ek and input data $\tilde{D} \in \{0,1\}^*$ with size up to $O(\mathsf{poly}(\lambda))$. $\mathsf{Preproc}$ outputs the preprocessed data D, and a *publicly verifiable* data tag τ_D (which may include the data size $N = |D|$ and potentially a commitment to D).

Remark 2 (Secret-keyed data preprocessing). Our formalization of data preprocessing closely follows that of [19][11] (Section 2.2) but is specified to the secret-key setting. Concretely, a client entity produces D by applying a rate-δ adversarial erasure code [9] to $\tilde{D}$ using a secret encoding key ek. For a multi-client entity, ek may be generated distributively via a distributed key generation mechanism [24].

Consequently, D is resilient to arbitrary adversarial deletions of up to δ fraction of its blocks, allowing $\tilde{D}$ to be recovered from D after such deletion. As mentioned in [19], this approach addresses the retrievability of the original input data $\tilde{D}$ without affecting: (1) the retrievability of D, which relates to our fairness property; and (2) the security of verifiable data replication, which is fundamental to our adaptive replication soundness. Therefore, we will describe the remaining algorithms and security definitions of our protocol *w.r.t.* the preprocessed, erasure-resilient data D, without explicitly revisiting $\mathsf{Preproc}$ each time.

The remaining phases, *i.e.*, Delegation, Preservation, and Retrieval, are executed between a client entity $\mathcal{C}$ and potentially a group of servers. We present their syntax as one-on-one executions between $\mathcal{C}$, identified by $(sk_{\mathcal{C}}, pk_{\mathcal{C}})$, and each server $\mathcal{S}$, identified by $(sk_{\mathcal{S}}, pk_{\mathcal{S}})$, of the group. Moreover, we assume, *w.l.o.g.*, that $\mathcal{C}$ and $\mathcal{S}$ respectively possess (*i.e.*, know the redeemer for) a payment output out_{pay} and a collateral output out_{col} from confirmed transactions $\mathsf{tx}_{pay}, \mathsf{tx}_{col} \in \mathcal{L}$. We specify these outputs as $\mathsf{out}_{pay} = (\nu_{pay}, p, \delta_{pay})$ and $\mathsf{out}_{col} = (\nu_{col}, c, \delta_{col})$, where:[12]

- ν_{pay} is the signature verification script for $pk_{\mathcal{C}}$ (*resp.*, ν_{col} for $pk_{\mathcal{S}}$);
- p is the payment value from $\mathcal{C}$, and c is the collateral value from $\mathcal{S}$; both are intended to be locked in the DSN contract through subsequent transactions;
- $\delta_{pay}, \delta_{col}$ are not specified, as they are external to the DSN contract.

Delegation phase. Let $\mathsf{md} := (id, n, V)$ denote a contract metadata that both parties mutually agree upon at the beginning of a delegation phase. Here, id is a *distinct* contract identifier derived from $pk_{\mathcal{C}}$ and $pk_{\mathcal{S}}$; n represents a contract period, specifying the number of test transactions that $\mathcal{S}$ should submit during the preservation phase; and $V = \{vi_{con}, vi_{test}, vi_{fin}\}$ denotes the validity intervals

[11] In fact, the model in [19] is general enough to also encompass encrypted data format used in [46].

[12] In terms of notation, we adopt the convention that, for all transactions, subscripts and superscripts are inherited component-wise from the enclosing transaction.

assigned to each transaction type, parameterized by the testing interval $T \in \mathsf{pp}$. The delegation phase consists of two interactive protocols ($\mathsf{OffDel}, \mathsf{OnDel}$), along with three deterministic algorithms ($\mathsf{TagVrfy}, \mathsf{DelVrfy}, \mathsf{Extract}$) that support the execution of the protocol and the formalization of the designed security.

- $\mathsf{OffDel}(\mathsf{md}, \langle \mathcal{C}(D, \tau_D), \mathcal{S} \rangle)$ takes as input a mutually agreed contract metadata $\mathsf{md} = (id, n, V)$ from both parties, and a preprocessed data D and its corresponding data tag τ_D from $\mathcal{C}$. It outputs:
  - to $\mathcal{S}$, a replica R and its corresponding replication auxiliary aux;
  - to $\mathcal{C}$, the replication auxiliary aux;

  The output of OffDel is denoted by $\langle \mathcal{C}(m_{con}), \mathcal{S}(R, m_{con}) \rangle$, where $m_{con} = (\mathsf{md}, \tau_D, \mathsf{aux})$ is the contact message;
- $\mathsf{TagVrfy}(D, \tau_D)$ outputs 1 if D is consistent with τ_D; or 0, otherwise;
- $\mathsf{DelVrfy}(id, \tau_D, \mathsf{aux})$ outputs 1 if aux is valid *w.r.t.* (id, τ_D); or 0, otherwise;
- $\mathsf{Extract}(id, \tau_D, \mathsf{aux}, R)$ outputs D extracted from R if (id, D, R) is consistent with (τ_D, aux); or $\perp$, otherwise.

To establish on-chain evidence, $\mathcal{C}$ and $\mathcal{S}$ execute OnDel to issue a contract transaction tx_{con} that embeds m_{con}. The on-chain evidence is as follows.

- $\mathsf{OnDel}(m_{con}, \mathsf{tx}_{pay}, \mathsf{tx}_{col}, \langle \mathcal{C}(sk_{\mathcal{C}}), \mathcal{S}(sk_{\mathcal{S}}) \rangle)$ takes as input a non-empty contract message $m_{con} \neq \perp$, two confirmed transactions $\mathsf{tx}_{pay}, \mathsf{tx}_{col} \in \mathcal{L}$, and $\mathcal{C}$ and $\mathcal{S}$'s local inputs $sk_{\mathcal{C}}$ and $sk_{\mathcal{S}}$. OnDel outputs a contract transaction tx_{con} to $\Pi_{\mathcal{L}}$ representing the agreement of $\mathcal{C}$ and $\mathcal{S}$. Parse $\mathsf{out}_{pay} = (\nu_{pay}, p, \delta_{pay})$ and $\mathsf{out}_{col} = (\nu_{col}, c, \delta_{col})$, tx_{con} consists of:
  - Inputs in_{con}^{pay}=$(\mathsf{out}_{pay}, \rho_{con}^{pay})$ and in_{con}^{col}=$(\mathsf{out}_{col}, \rho_{con}^{col})$;
  - Outputs $\mathsf{out}_{con}^{pay} = (\nu_{con}^{pay}, p, \delta_{con}^{pay})$ and out_{con}^{col}=$(\nu_{con}^{col}, c, \delta_{con}^{col})$ *s.t.* m_{con} is included in both δ_{con}^{pay} and δ_{con}^{col};
  - Validity interval vi_{con} obtained from $V \in \mathsf{md}$.

Following Eq. 1, the eligibility check of tx_{con} to spend tx_{pay} and tx_{col} is expressed, with datum omitted, as $\nu_{pay}(\rho_{con}^{pay}, \mathsf{tx}_{con}) \wedge \nu_{col}(\rho_{con}^{col}, \mathsf{tx}_{con})$. The confirmation of tx_{con}, *i.e.*, when $\mathsf{tx}_{con} \in \mathcal{L}$, marks the end of the delegation phase. From this point onward, $\mathcal{C}$ may delete her local $\tilde{D}$ and D, while $\mathcal{S}$ stores R. Neither party needs to retain m_{con}, as it is recorded[13] in $\mathsf{tx}_{con} \in \mathcal{L}$.

We now explain the outputs of tx_{con}. The *collateral* locked in out_{con}^{col} is spend by the first test transaction tx_{test}^{1}, which in turn re-locks the collateral for subsequent test transactions. Specifically, ν_{con}^{col} invokes the $\mathsf{TestVrfy}^1$ algorithm (defined in the preservation phase) *w.r.t.* a challenge ch^1. As discussed in Remark 1, ch^1 is publicly derived from tx_{con} with ν_{con}^{col} removed, *i.e.*, from $\mathsf{tx}_{con} \backslash \{\nu_{con}^{col}\}$, to avoid circular dependency. A similar method is employed when generating subsequent preservation challenges. The concrete processes are detailed in the construction to be presented in Section 4.2.

In contrast, the consumption of the locked *payment* is left unspecified. It can be configured to be spent by $\mathcal{S}$, *e.g.*, entirely upfront (*i.e.*, full payment)

[13] The blockchain storage overhead incurred by storing such off-chain data is briefly discussed in Remark 4.

or incrementally over the contract period (*i.e.*, installments). This configuration choice mainly concerns the incentive model and rational behavior of participants, rather than the security of our DSN protocol.

Finally, to ensure contract continuity, we require all transactions to embed the contract message m_{con} in their output datums, and for their validity intervals to conform to $V \in \mathsf{md}$. This consistency is also enforced by the eligibility check.

Preservation phase. Parse the contract metadata $\mathsf{md} \coloneqq (id, n, V)$. This phase is structured as a sequence of indexed procedures $\{\mathsf{Test}^i, \mathsf{TestVrfy}^i\}_{i\in[n]}$. Put $\mathsf{tx}^0_{test} \coloneqq \mathsf{tx}_{con}$, the syntax is defined inductively for $i \in [n]$.

- $\mathsf{Test}^i(\mathsf{tx}^{i-1}_{test}, R)$, run by $\mathcal{S}$, takes as input the latest confirmed test transaction $\mathsf{tx}^{i-1}_{test} \in \mathcal{L}$ (detailed below for index i) and the local replica R corresponding to the contract message $m_{con} = (\mathsf{md}, \tau_D, \mathsf{aux})$ obtained from tx^{i-1}_{test}. It derives the challenge ch^i from the public seed, denoted by $\mathsf{seed}^{i-1} \coloneqq \mathsf{tx}^{i-1}_{test} \backslash \{\nu^{i-1}_{test}\}$. Test^i outputs a new test transaction tx^i_{test} to $\Pi_{\mathcal{L}}$, embedding the proof in response to ch^i, denoted by π^i. tx^i_{test} consists of:
  - Input $\mathsf{in}^i_{test} = (\mathsf{out}^{i-1}_{test}, \rho^i_{test})$ *s.t.* $(\mathsf{ch}^i, \pi^i) \in \rho^i_{test}$;
  - Output $\mathsf{out}^i_{test} = (\nu^i_{test}, c, \delta^i_{test})$ that locks the collateral *s.t.* ν^i_{test} invokes[14] $\mathsf{TestVrfy}^{i+1}$ *w.r.t.* ch^{i+1} (derived from $\mathsf{seed}^i = \mathsf{tx}^i_{test} \backslash \{\nu^i_{test}\}$), and $m_{con} \in \delta^i_{test}$. Particularly, when $i = n$, ν^n_{test} is deferred to the retrieval phase;
  - Validity interval $vi^i_{test} = vi_{test}$ obtained from $V \in \mathsf{md}$.
- $\mathsf{TestVrfy}^i(id, \tau_D, \mathsf{aux}, \mathsf{ch}^i, \pi^i, \mathsf{seed}^{i-1})$ is deterministic. It takes as input a contract identifier id, a data tag τ_D, replication auxiliary aux, and the challenge-proof-seed tuple $(\mathsf{ch}^i, \pi^i, \mathsf{seed}^{i-1})$. $\mathsf{TestVrfy}^i$ outputs 1 if: (1) ch^i is correctly derived from seed^{i-1}, and (2) π is a valid proof *w.r.t.* $(id, \tau_D, \mathsf{aux}, \mathsf{ch}^i)$; otherwise, it outputs 0.

The eligibility of tx^i_{test} to spend tx^{i-1}_{test} is given by $\nu^{i-1}_{test}(\delta^{i-1}_{test}, \rho^i_{test}, \mathsf{tx}^i_{test})$. In addition to invoking $\mathsf{TestVrfy}^i$, ν^{i-1}_{test} checks that the same m_{con} is embedded in both δ^{i-1}_{test} and δ^i_{test} to ensure contract continuity. For authentication, it may further require a signature from $\mathcal{S}$. The confirmation of the final test transaction, *i.e.*, $\mathsf{tx}^n_{test} \in \mathcal{L}$, marks the end of the preservation phase.

As an optional scalability optimization, recursive proof systems [6, 8] can be employed to reduce the number of test transactions recorded on the blockchain.

Retrieval phase. $\mathcal{S}$ initiates the retrieval phase by sending $\mathcal{C}$ a *concealed* version of her replica. In return, $\mathcal{S}$ receives a receipt (*i.e.*, in the form of a partial transaction) that can be completed upon revealing the secret for concealment. This allows $\mathcal{C}$ to retrieve her data and $\mathcal{S}$ to reclaim the collateral. The process aligns with the design of existing (E)UTxO-based fair exchange protocols. To precisely define our enhanced fairness, we explicitly structure the retrieval phase into four components: $(\mathsf{InitRet}, \mathsf{RetVrfy}, \mathsf{Finalize}, \mathsf{RetExt})$, where $\mathsf{InitRet}$ is an interactive protocol, and the remaining three are deterministic algorithms.

[14] This includes the base case $i = 0$, where $\nu^0_{test} \coloneqq \nu^{col}_{con}$ invokes $\mathsf{TestVrfy}^1$ as explained.

– $\mathsf{InitRet}(\mathsf{tx}^n_{test}, \langle \mathcal{C}(sk_{\mathcal{C}}), \mathcal{S}(R, w)\rangle)$ takes as input the confirmed $\mathsf{tx}^n_{test} \in \mathcal{L}$, $\mathcal{C}$'s local input $sk_{\mathcal{C}}$, and $\mathcal{S}$'s local replica R (corresponding to the contract message $m_{con} = (\mathsf{md}, \tau_D, \mathsf{aux})$ obtained from tx^n_{test}), and a witness key w that is internally and freshly generated for each retrieval session, we write it explicitly in the input for security analysis. It outputs:
  • to $\mathcal{C}$, a concealed replica C and its corresponding retrieval auxiliary $\mathsf{aux}_{\mathsf{ret}}$;
  • to $\mathcal{S}$, a *partial* transaction $\hat{\mathsf{tx}}_{fin}$ that consists of:
    ∗ Partial input $\hat{\mathsf{in}}_{fin} = (\mathsf{out}^n_{test}, \hat{\rho}_{fin})$ *s.t.* $\mathsf{aux}_{\mathsf{ret}} \in \hat{\rho}_{fin}$;
    ∗ Output $\mathsf{out}_{fin} = (\nu_{fin}, c, \delta_{fin})$ that locks the collateral *s.t.* ν_{fin} invokes $\mathsf{SIG.Vrfy}$ *w.r.t.* $pk_{\mathcal{S}}$[15], and $m_{con} \in \delta_{fin}$;
    ∗ Validity interval vi_{fin} obtained from $V \in \mathsf{md}$.
  The output of $\mathsf{InitRet}$ is denoted by $(C, \mathsf{aux}_{\mathsf{ret}}, \hat{\mathsf{tx}}_{fin})$;
– $\mathsf{RetVrfy}(id, \tau_D, \mathsf{aux}, C, \mathsf{aux}_{\mathsf{ret}})$ is defined independent of transactions. It outputs 1 if $(C, \mathsf{aux}_{\mathsf{ret}})$ is valid *w.r.t.* $(id, \tau_D, \mathsf{aux})$, or 0 otherwise.

The well-formedness of $\hat{\mathsf{tx}}_{fin}$ is given by $\mathcal{O}_{\mathsf{txVrfy}}$. We further *specify* that $\mathcal{O}_{\mathsf{txVrfy}}(\hat{\mathsf{tx}}_{fin}) = 0$ if the current round is less then u (*i.e.*, the liveness delay) rounds away from the *right* boundary of the agreed validity interval vi_{fin}. This restriction prevents the "withholding attack" discussed in Remark 3. Moreover, we let ν^n_{test} to also validate $(\hat{\rho}_{fin}, \hat{\mathsf{tx}}_{fin})$, which serves as a subprocess within the validation of tx_{fin} (as detailed below, ρ_{fin} extends directly atop $\hat{\rho}_{fin}$). Formally, we write, with the abbreviated notation, $\nu^n_{test}(\hat{\rho}_{fin}) = 1$ if $(\hat{\rho}_{fin}, \hat{\mathsf{tx}}_{fin})$ is valid *w.r.t.* ν^n_{test}, or 0 otherwise.

Remark 3 (Partial transaction withholding attack). A malicious client $\mathcal{C}$ may deliberately withhold $\hat{\mathsf{tx}}_{fin}$ from an honest $\mathcal{S}$ until there is barely enough time to adapt and submit tx_{fin} within the predefined validity interval vi_{fin}, but insufficient time for tx_{fin} to be processed on-chain due to liveness delays. Nevertheless, a published (even if unconfirmed) tx_{fin} enables $\mathcal{C}$ to extract the witness key and retrieve the data, while $\mathcal{S}$ risks losing her collateral. This violates the fairness guarantee intended for honest servers. In contrast, our security definition (Definition 9) is scoped to only confirmed transactions, *i.e.*, $\mathsf{tx}_{fin} \in \mathcal{L}$, and hence rules out this type of attack. To mitigate it at the protocol level, we incorporate an additional restriction into $\mathcal{O}_{\mathsf{txVrfy}}$ to bound the window for client withholding.

The remainder, *i.e.*, $(\mathsf{Finalize}, \mathsf{RetExt})$, allows $\mathcal{S}$ to complete a *valid* partial transaction using her secret witness w, and enables $\mathcal{C}$ to retrieve her data from the full transaction. The algorithms are performed as follows.

– $\mathsf{Finalize}(w, \hat{\mathsf{tx}}_{fin})$, run by $\mathcal{S}$, takes as input a witness key w and a partial transaction $\hat{\mathsf{tx}}_{fin}$. It outputs the full finalization transaction tx_{fin} to $\Pi_{\mathcal{L}}$ by adapting the partial redeemer to ρ_{fin} *s.t.* $(\hat{\rho}_{fin}, w) \in \rho_{fin}$;
– $\mathsf{RetExt}(C, \mathsf{tx}_{fin})$, run by $\mathcal{C}$, takes as input a concealed value C and a finalization transaction tx_{fin}. It outputs a tuple (w, R, D) *s.t.* the witness key w is extracted from $(\hat{\rho}_{fin}, \rho_{fin})$, the replica R is extracted from (C, w), and the data D is extracted from $(id, \tau_D, \mathsf{aux}, R)$ by inherently invoking $\mathsf{Extract}$.

[15] That is, $\mathcal{S}$ can reclaim her collateral by submitting a transaction, signed under $sk_{\mathcal{S}}$, that spends out_{fin} of $\mathsf{tx}_{fin} \in \mathcal{L}$.

For convenience, we may overload the notation RetExt and define the intermediate extraction step as $(R, D) \leftarrow \mathsf{RetExt}(C, w)$.

The eligibility of tx_{fin} to spend tx^n_{test} is determined by $\nu^n_{test}(\delta^n_{test}, \rho_{fin}, \mathsf{tx}_{fin})$, where the relation of $\mathsf{aux}_{\mathsf{ret}} \in \hat{\rho}_{fin}$ and w is examined in addition to $\nu^n_{test}(\hat{\rho}_{fin})$. At the end of the protocol, $\mathcal{C}$ retrieves her original input data $\tilde{D}$ by decoding D with ek (recall Remark 2). Concurrently, $\mathcal{S}$ can spend the collateral locked in out_{fin} by using her $sk_{\mathcal{S}}$ to redeem ν_{fin}.

Further retrieval policies and configurations are deferred to Appendix C.1, and privacy considerations regarding the publicly exposed witness key w are discussed in Appendix C.2.

Remark 4 (Blockchain storage overhead). We consider data D (and $\tilde{D}$), replica R, and the concealed replica C to be "big", and hence should never be recorded on $\mathcal{L}$. Under our current syntax, $\mathcal{L}$ only records the contract message m_{con} through datums across all transactions, along with essential proofs embedded in redeemers and validation scripts attached to validators. Redundancy can be further reduced by storing m_{con} only in tx_{con} and referencing its hash in subsequent transactions. Additionally, techniques mentioned in Footnote 7 can be applied to improve memory efficiency during processing.

Full protocol depiction. We provide a pictorial depiction in Figure 2.

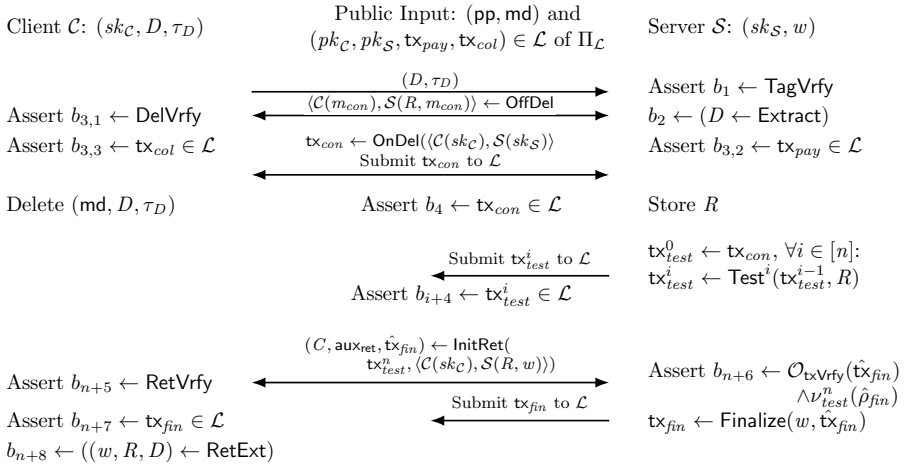


Fig. 2: An honest execution of DSN between client entity $\mathcal{C}$ and server $\mathcal{S}$.

An asserting verification detects a failure that, if encountered, terminates the protocol with $\perp$. In contrast, non-asserting verifications are used to define correctness without enforcing an immediate halt, *e.g.*, in verification b_2, $\mathcal{C}$ cannot determine whether D is extractable from R, as R is solely produced by $\mathcal{S}$

(hence b_2 is placed on the server side). Instead, $\mathcal{C}$ only learns probabilistically, via verification $b_{3,1}$ through $\mathsf{DelVrfy}$, whether $\mathcal{S}$ has honestly generated R.

We further recall that confirmation of a transaction inherently implies its validity (*i.e.*, well-formedness and eligibility to spend the referred outputs). Hence, we assert transaction confirmation before advancing to the next phase and during the progression within the preservation phase, *i.e.*, via b_4 and b_{n+4} for $i \in [n]$. These verifications are deliberately placed in the middle to indicate that they are enforced directly on-chain by the EUTxO validation logic. Another notable case is b_{n+6}, b_{n+7}, which relates to the withholding attack discussed in Remark 3. While $\mathcal{C}$ may extract w without a confirmed tx_{fin}, the asserted b_{n+6} leverages the specified well-formedness oracle, *i.e.*, $\mathcal{O}_{\mathsf{txVrfy}}(\hat{\mathsf{tx}}_{fin})$, to ensure that if an honest $\mathcal{S}$ executes $\mathsf{Finalize}$ over a valid $\hat{\mathsf{tx}}_{fin}$, then tx_{fin} must be confirmable on $\mathcal{L}$. This subsumes the validity check into the confirmation check at b_{n+7}.

3.3 Security Definitions

This section presents the formal definitions of correctness and security properties. Table 1 summarizes the notations introduced in our protocol syntax.

Table 1: Notation Table

Notation	Description	Notation	Description
pp	public parameter	$\tilde{D}$	original input data
D	preprocessed data	τ_D	data tag
$(sk_{\mathcal{C}}, pk_{\mathcal{C}})$	the key pair identifying $\mathcal{C}$ in $\Pi_{\mathcal{L}}$	$(sk_{\mathcal{S}}, pk_{\mathcal{S}})$	the key pair identifying $\mathcal{S}$ in $\Pi_{\mathcal{L}}$
tx_{pay}	a transaction including $\mathcal{C}$'s payment output	tx_{col}	a transaction including $\mathcal{S}$'s collateral output
id	contract identifier	n	contract period
V	validity intervals	md	contract metadata $\mathsf{md} = (id, n, V)$
R	replica	aux	replication auxiliary
m_{con}	contract message $m_{con} = (\mathsf{md}, \tau_D, \mathsf{aux})$	ch^i	the i-th challenge
π^i	the i-th proof	w	retrieval witness key
C	concealed replica	$\mathsf{aux}_{\mathsf{ret}}$	retrieval auxiliary
tx_{con}	delegation transaction	tx^i_{test}	the i-th test transaction
$\hat{\mathsf{tx}}_{fin}$	partial retrieval transaction	tx_{fin}	retrieval transaction

Correctness. We start with the experiment for correctness, *i.e.*, $\mathsf{Expt}^{\mathrm{correct}}_{\mathsf{DSN},\mathcal{C},\mathcal{S}}(\mathsf{md}, D, \tau_D, w)$, as given in Figure 3. The verifications, including their indices, are aligned with those depicted in Figure 2, with one exception: the sequence ($b_{3,1}$, $b_{3,2}, b_{3,3}$) is merged into a single verification b_3 for notational simplicity.

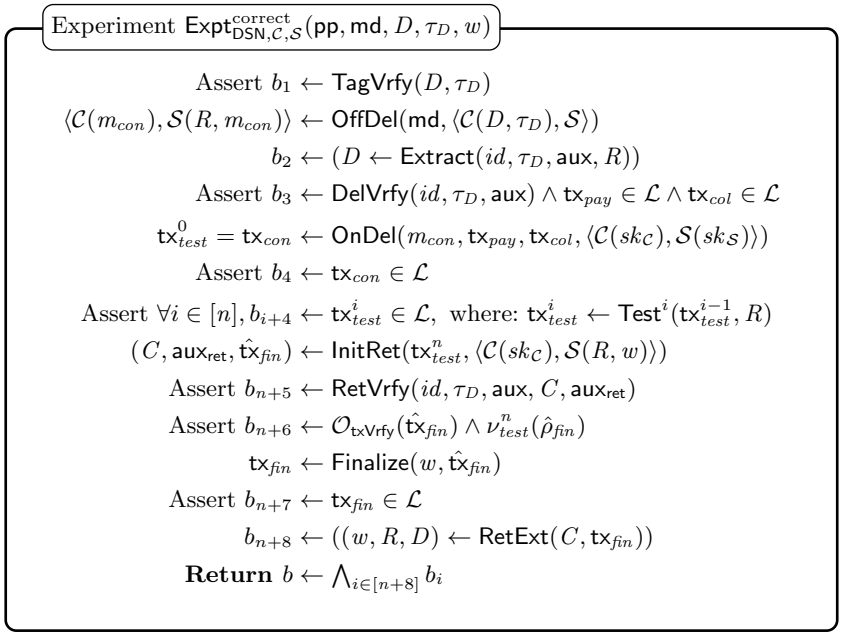


Fig. 3: The experiment for correctness. Both $\mathcal{C}$ and $\mathcal{S}$ receive the public parameter pp (omitted in the main body) and execute the protocol with $(\mathsf{md}, D, \tau_D, w)$. Additionally, they implicitly provide the respective inputs $(sk_{\mathcal{C}}, pk_{\mathcal{C}}, \mathsf{tx}_{pay})$ and $(sk_{\mathcal{S}}, pk_{\mathcal{S}}, \mathsf{tx}_{col})$.

Definition 4 (Correctness). *A blockchain-aided DSN protocol is correct if, for any $\lambda > 0$, any pair of client $\mathcal{C}$ and server $\mathcal{S}$ possessing $(sk_{\mathcal{C}}, pk_{\mathcal{C}}, \mathsf{tx}_{pay})$, $(sk_{\mathcal{S}}, pk_{\mathcal{S}}, \mathsf{tx}_{col})$, respectively, and for any contract metadata md, preprocessed data-tag pair (D, τ_D), and witness key w, there exists t_λ s.t. the following probability holds for any $t \geq t_\lambda$ and $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, t)$*

$$\Pr\left[\mathsf{Expt}^{\mathrm{correct}}_{\mathsf{DSN},\mathcal{C},\mathcal{S}}(\mathsf{pp},\mathsf{md},D,\tau_D,w)=1\right]=1.$$

Delegation binding and contract unforgeability. Correctness ensures, via verification b_2, that if a *valid* data-tag pair (D, τ_D) is used as input to OffDel to produce (R, aux), then D can be extracted from $(id, \tau_D, \mathsf{aux}, R)$. However, as discussed in [19], meaningful extractability requires that (id, D, R) be bound to (τ_D, aux); otherwise, a client may fail to recover a consistent D. We formalize this requirement as *delegation binding*, with its experiment provided in Figure 4.

Definition 5 (Delegation Binding). *A blockchain-aided DSN protocol has binding (off-chain) delegation if, for any PPT adversary $\mathcal{A}$, the following probability is $\mathsf{negl}(\lambda)$ for any $\lambda > 0$ and $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \cdot)$*

$$\Pr\left[\mathsf{Expt}^{\text{del-bind}}_{\mathsf{DSN},\mathcal{A}}(\mathsf{pp})=1\right].$$

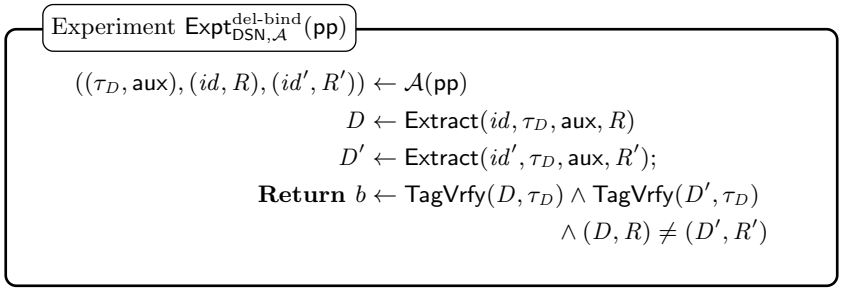


Fig. 4: The experiment for delegation binding. The adversary $\mathcal{A}$ receives pp and is required to output a tuple $((\tau_D, \mathsf{aux}), (id, R), (id', R'))$ *s.t.* both $(id, \tau_D, \mathsf{aux}, R)$ and $(id', \tau_D, \mathsf{aux}, R')$ extract to valid data, and the resulting data-replica pairs are distinct.

On the other hand, contract unforgeability ensures that no adversary $\mathcal{A}$, impersonating either a malicious $\mathcal{C}$ or $\mathcal{S}$, can unilaterally produce a valid contract transaction on behalf of its honest counterparty, even when given oracle access to interactively execute OnDel with that counterparty. We define the oracles below, and provide the experiment in Figure 5.

– $\mathcal{O}_{\mathsf{CliOnDel}}(\mathsf{pp}, sk_{\mathcal{C}}, \mathsf{tx}_{pay}, \cdot)$ (*resp.*, $\mathcal{O}_{\mathsf{SerOnDel}}(\mathsf{pp}, sk_{\mathcal{S}}, \mathsf{tx}_{col}, \cdot)$). On query $(m, sk_{\mathcal{A}}, \mathsf{tx}_{\mathcal{A}})$, it returns $\perp$ if $\mathcal{O}_{\mathsf{txVrfy}}(\mathsf{tx}_{\mathcal{A}}) = 0$. Otherwise, $\mathcal{O}_{\mathsf{CliOnDel}}$ (*resp.*, $\mathcal{O}_{\mathsf{SerOnDel}}$) proceeds with $\mathcal{A}$ impersonating a malicious server (*resp.*, client):

$$\mathsf{tx}_{con} \leftarrow \mathsf{OnDel}(m, \mathsf{tx}_{pay}, \mathsf{tx}_{\mathcal{A}}, \langle \mathcal{C}(sk_{\mathcal{C}}), \mathcal{A}(sk_{\mathcal{A}}) \rangle).$$
$$(\textit{resp.},\ \mathsf{tx}_{con} \leftarrow \mathsf{OnDel}(m, \mathsf{tx}_{\mathcal{A}}, \mathsf{tx}_{col}, \langle \mathcal{A}(sk_{\mathcal{A}}), \mathcal{S}(sk_{\mathcal{S}}) \rangle)$$

The oracle returns tx_{con} and records $\mathsf{tx}_{con} \backslash \{\rho^{pay}_{con}\}$ (*resp.*, $\mathsf{tx}_{con} \backslash \{\rho^{col}_{con}\}$) in the queried set Q_{CliOnDel} (*resp.*, Q_{SerOnDel}).

Definition 6 (Contract Unforgeability). *A blockchain-aided DSN protocol has unforgeable contract if, for any PPT adversary $\mathcal{A}$ with access to oracles $\mathcal{O}_{\mathsf{CliOnDel}}$ and $\mathcal{O}_{\mathsf{SerOnDel}}$, the following probability is $\mathsf{negl}(\lambda)$ for any $\lambda > 0, \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \cdot)$, $(sk_{\mathcal{C}}, pk_{\mathcal{C}}), (sk_{\mathcal{S}}, pk_{\mathcal{S}})$ from $\mathsf{SIG.KGen}(1^\lambda)$, and any $\mathsf{tx}_{pay}, \mathsf{tx}_{col} \in \mathcal{L}$,*

$$\Pr\left[\mathsf{Expt}^{\text{del-unforge}}_{\mathsf{DSN},\mathcal{A}}(\mathsf{pp}, pk_{\mathcal{C}}, pk_{\mathcal{S}}, \mathsf{tx}_{pay}, \mathsf{tx}_{col}) = 1\right].$$

Combining contract unforgeability with correctness throughout the delegation phase establishes the *atomicity* of delegation, *i.e.*, $\mathsf{tx}_{con} \in \mathcal{L}$ if and only if both parties agree on a valid contract message m_{con}. Furthermore, delegation binding ensures that for any *valid* m_{con}, the associated replica R is uniquely extractable, *i.e.*, $\mathsf{Extract}$ outputs exact the delegated D when given R and $(id, \tau_D, \mathsf{aux}) \in m_{con}$.

However, this does not guarantee that a malicious server will continue storing the same R, or even a not-too-compressed format of R that remains extractable

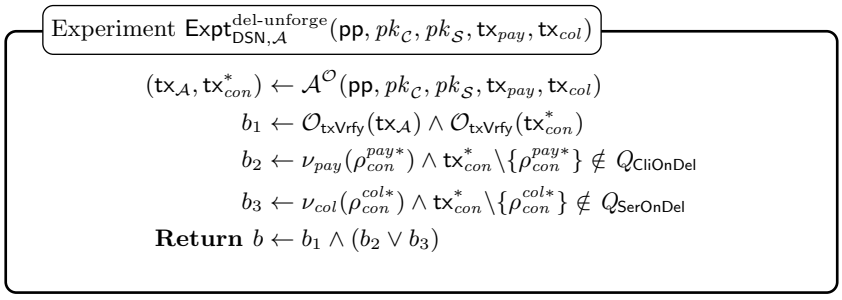


Fig. 5: The experiment for contract unforgeability. The adversary $\mathcal{A}$, who inherently holds a *valid* key pair $(sk_{\mathcal{A}}, pk_{\mathcal{A}}) \leftarrow \mathsf{SIG.KGen}(1^{\lambda})$, receives pp, along with valid public keys $pk_{\mathcal{C}}, pk_{\mathcal{S}}$ and transactions $\mathsf{tx}_{pay}, \mathsf{tx}_{col} \in \mathcal{L}$. It may generate a transaction $\mathsf{tx}_{\mathcal{A}}$ of its choice, and is granted access to oracles $\mathcal{O} \coloneqq \{\mathcal{O}_{\mathsf{CliOnDel}}, \mathcal{O}_{\mathsf{SerOnDel}}\}$, as defined above. $\mathcal{A}$ wins if it outputs a well-formed tx^*_{con} *s.t.* at least one redeemer in tx^*_{con}, denoted by ρ^{pay*}_{con} (*resp.*, ρ^{col*}_{con}), is valid *w.r.t.* to $\nu_{pay} \in \mathsf{tx}_{pay}$ (*resp.*, $\nu_{col} \in \mathsf{tx}_{col}$). Moreover, the underlying transaction body, *i.e.*, tx^*_{con} without ρ^{pay*}_{con} or ρ^{col*}_{con}, must not have been queried to the respective oracle.

after the delegation phase, especially when the server is assigned, or colludes with those who are assigned, multiple copies of the same D under distinct identifiers. Similar requirements are formulated under replication soundness [19], but as noted in Section 1.1, the existing definition fails to capture scenarios where proofs are generated iteratively and stored externally, as in our blockchain-aided DSN protocol. This limitation motivates us to revisit replication soundness in an adaptive setting. Moreover, since the impossibility result from [19] (discussed in Section 2.2) also applies here, we define our adaptive replication soundness under a similar rational adversary model.

Adaptive replication soundness. The starting point is the non-adaptive replication soundness experiment (Figure 1) in Section 2.2. To extend it to the adaptive setting, a straightforward approach is to organize the input data vectors and their corresponding iterative challenge-response processes into a matrix structure. However, this fails to accurately reflect the preservation phase of the DSN protocol in practice, where different test sessions (*i.e.*, Test^i, for all $i \in [n]$) may proceed at different speeds, making index alignment across sessions infeasible. We address this by focusing the model on a single target data item, and granting the adversary oracle access to proofs from parallel test sessions.

Specifically, the adversary is enhanced in two ways: (1) it has access to all previously generated proofs, and the current internal state, which is updated before each new iteration; and (2) it is granted access to a simulation-test oracle $\mathcal{O}_{\mathsf{SimTest}}$ that returns proofs generated on potentially identical input data. The first enhancement captures an attack, in which the adversary references previously published proofs on the blockchain and adjusts its internal storage

accordingly. The second models an attack based on the malleability of proofs, where $\mathcal{O}_{\mathsf{SimTest}}$ is similar to the oracle in the simulation-extractability definition (Definition 20). We define $\mathcal{O}_{\mathsf{SimTest}}$ along with a helper oracle, *i.e.*, $\mathcal{O}_{\mathsf{CliDelPh}}$, which enables $\mathcal{A}$ to generate valid tx_{con}. The experiment for our adaptive replication soundness in provided Figure 6.

- $\mathcal{O}_{\mathsf{CliDelPh}}(\mathsf{pp}, sk_{\mathcal{C}}, \cdot)$. On query (md, D, τ_D), it returns $\perp$ if $\mathsf{TagVrfy}(D, \tau_D) = 0$. Otherwise, $\mathcal{O}_{\mathsf{CliDelPh}}$ performs the delegation phase with $\mathcal{A}$ impersonating a malicious server[16], and returns (R, tx_{con}) to $\mathcal{A}$, *s.t.* $\mathsf{tx}_{con} \in \mathcal{L}$;
- $\mathcal{O}_{\mathsf{SimTest}}(\mathsf{pp}, \mathsf{td}, \cdot)$. On query m_{con}=$(\mathsf{md}, \tau_D, \mathsf{aux})$, it returns $\perp$ if $\mathsf{DelVrfy}(id, \tau_D, \mathsf{aux}) = 0$. Otherwise, $\mathcal{O}_{\mathsf{SimTest}}$ *honestly* executes OnDel to obtain tx_{con}, and simulates the proof generation using the trapdoor td, *i.e.*, returning $\{\pi^i\}_{i\in[n]}$ to $\mathcal{A}$, where each π^i is a simulated proof for the challenge ch^i derived from $\mathsf{tx}^{i-1}_{test}\backslash\{\nu^{i-1}_{test}\}$, and each tx^i_{test} is the output of $\mathsf{Test}^i(\mathsf{tx}^{i-1}_{test}, \cdot)$, with $\mathsf{tx}^0_{test} = \mathsf{tx}_{con}$. Finally, $\mathcal{O}_{\mathsf{SimTest}}$ records $(m_{con}, \{\pi^i\}_{i\in[n]})$ in the queried set Q_{SimTest}.

Experiment $\mathsf{Expt}^{\text{adapt-rep-sound}}_{\mathsf{DSN},\mathcal{A},\mathcal{E}}(\mathsf{pp})$

$$
\begin{aligned}
(\mathsf{md}, D, \tau_D) &\leftarrow \mathcal{A}_0(\mathsf{pp}) \ \textit{s.t.}\ \mathsf{TagVrfy}(D, \tau_D) = 1\\
(R, \mathsf{tx}_{con}) &\leftarrow \mathcal{A}_0^{\mathcal{O}_{\mathsf{CliDelPh}}}(\mathsf{pp}, \mathsf{md}, D, \tau_D) \ \textit{s.t.}\ \mathsf{tx}_{con} \in \mathcal{L}\\
\mathsf{st}^0 &\leftarrow R, \mathsf{tx}^0_{test} \leftarrow \mathsf{tx}_{con}\\
\forall i \in [n], (\mathsf{st}^i, \mathsf{tx}^i_{test}) &\leftarrow \mathcal{A}_i^{\mathcal{O}_{\mathsf{SimTest}}}(\mathsf{pp}, \mathsf{st}^{i-1}, \{\pi^j\}_{j\in[i-1]_0})\\
\{R^{i-1}\}_{i\in[n]} &\leftarrow \mathcal{E}^{\{\mathcal{A}_i\}_{i\in[n]}}(\mathsf{pp}, \mathsf{tx}_{con}, Q_{\mathsf{SimTest}})\\
\textbf{Return}\ \ &\{(\mathsf{tx}^i_{test}, \mathsf{st}^{i-1}, R^{i-1})\}_{i\in[n]}
\end{aligned}
$$

Fig. 6: The experiment for adaptive replication soundness. The adversary $\mathcal{A}_0 \in \mathcal{A}$ receives pp and outputs (md, D, τ_D) of its choice. It then interacts with $\mathcal{O}_{\mathsf{CliDelPh}}$ to obtain (R, tx_{con}). The set of subprocesses $\{\mathcal{A}_i\}_{i\in[n]}$ executes the preservation phase iteratively. That is, for all $i \in [n]$, $\mathcal{A}_i$ produces a test transaction tx^i_{test} that is stored externally (on $\mathcal{L}$) and updates its internal storage state st^i using the latest internal state st^{i-1} and the knowledge of all previously generated *proofs*. We assume the adversary only leverages the previous proofs; other transaction components, *e.g.*, datums, are not considered in this model. An extractor $\mathcal{E}^{\{\mathcal{A}_i\}_{i\in[n]}}$ is invoked to output all intermediate (potentially compressed) replicas. The experiment ends by returning $\{(\mathsf{tx}^i_{test}, \mathsf{st}^{i-1}, R^{i-1})\}_{i\in[n]}$.

Definition 7 (Adaptive Replication Soundness). *A blockchain-aided DSN protocol satisfies (ϵ_1, ϵ_2)-adaptive replication soundness if, for any adversary $\mathcal{A} = (\{\mathcal{A}_i\}_{i\in[n]_0})$, where $\mathcal{A}_0$ runs in $O(\mathsf{poly}(\lambda))$, and each $\mathcal{A}_i$, for $i \in [n]$, runs in*

[16] A confirmed collateral transaction $\mathsf{tx}_{col} \in \mathcal{L}$, possessed by $\mathcal{A}$, is *implicitly* provided in $\mathcal{A}$'s query, as it is irrelevant to the preservation phase.

parallel time at most $T(\lambda, t)$*, there exists an extractor* $\mathcal{E}^{\{\mathcal{A}_i\}_{i\in[n]}}$ *s.t.*

$$\Pr\left[\begin{array}{c} D \leftarrow \mathsf{Extract}(id, \tau_D, \mathsf{aux}, R^j) \\ \wedge |R^j| \geq (1-\epsilon_1)|R^{j-1}| \\ \wedge |\mathsf{st}^j| \geq (1-\epsilon_2)|R^j|, \forall j \in [i-1]_0 \end{array} \middle| \begin{array}{c} \{(\mathsf{tx}^i_{test}, \mathsf{st}^{i-1}, R^{i-1})\}_{i\in[n]} \\ \leftarrow \mathsf{Expt}^{\text{adapt-rep-sound}}_{\mathsf{PoRep},\mathcal{A},\mathcal{E}}(\mathsf{pp}) \end{array}\right]$$

is at least $1 - \mathsf{negl}(\lambda)$ *for any* $\lambda, t > 0$*,* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, t)$*, and any* $i \in [n]$ *s.t.* $\mathcal{O}_{\mathsf{txVrfy}}(\mathsf{tx}^i_{test}) \wedge \nu^{i-1}_{test}(\rho^i_{test}) = 1$.

The intuition of the probability formula is as follows: if any test transaction is deemed valid, the extractor must be able to successfully extract all previous replicas, and the size difference between any two consecutive extracted replicas must not exceed a compression factor of $1 - \epsilon_1$. Additionally, the adversary's internal state must retain at least $1 - \epsilon_2$ of the size of the extracted replica.

Adaptive client-fairness. At a high-level, a malicious server may break client-fairness in two ways: (1) by producing a concealed replica C and retrieval auxiliary $\mathsf{aux}_{\mathsf{ret}}$ that pass the $\mathsf{RetVrfy}$ verification, yet the witness (extracted by a knowledge extractor) fails to give a consistent replica for the honest client, hence preventing the extraction of data; or (2) by forging a valid finalization transaction tx_{fin} that unlocks the server's collateral but cannot be extracted into any meaningful witness key w, and thus fails to produce any valid replica or data. The existing client-fairness definition in [41] considers only a non-adaptive adversary who is assigned a replica R. We defer an analogous non-adaptive client-fairness definition under our DSN protocol syntax to Appendix C.2.

In contrast, an adaptive adversary that stores or colludes over multiple replicas of identical data may leverage (*e.g.*, via malleability) valid $\mathsf{aux}_{\mathsf{ret}}$ from other retrieval sessions to construct a tuple $(R, C, \mathsf{aux}_{\mathsf{ret}})$ maximizing its attack advantage. To address this, we enhance our security model to a strong adaptive setting, where the adversary $\mathcal{A}$ may choose the data D to be delegated and is additionally given access to an oracle $\mathcal{O}_{\mathsf{SimAux}}$ to capture the potential malleability attack. We also grant $\mathcal{A}$ access to oracles $\mathcal{O}_{\mathsf{CliRet}}, \mathcal{O}_{\mathsf{CliInitRet}}, \mathcal{O}_{\mathsf{NewAux}}$ that capture possible forgeries of tx_{fin}. The definitions of oracles are as follows. We further provide the experiment for adaptive client-fairness in Figure 7.

- $\mathcal{O}_{\mathsf{SimAux}}(\mathsf{pp}, \mathsf{td}, \cdot)$. On query $(id, \tau_D, \mathsf{aux}, C)$, it uses td to generate a simulated retrieval auxiliary $\mathsf{aux}_{\mathsf{ret}}$, returns it to $\mathcal{A}$, and records $(id, \tau_D, \mathsf{aux}, C, \mathsf{aux}_{\mathsf{ret}})$ in the queried set Q_{SimAux};
- $\mathcal{O}_{\mathsf{CliRet}}(\mathsf{pp}, sk_{\mathcal{C}}, \cdot)$. On query $(R, \mathsf{tx}^n_{test}, \mathsf{tx}_{fin} \backslash \{\rho_{fin}\})$, it returns $\perp$ if

$$(\perp \leftarrow \mathsf{Extract}(id, \tau_D, \mathsf{aux}, R)) \vee \mathsf{tx}^n_{test} \notin \mathcal{L}.$$

  Otherwise, $\mathcal{O}_{\mathsf{CliRet}}$ samples w and *honestly* executes $(\mathsf{InitRet}, \mathsf{Finalize})$ to obtain $(C, \mathsf{aux}_{\mathsf{ret}}, \hat{\mathsf{tx}}_{fin}, \mathsf{tx}_{fin})$, where $\hat{\mathsf{tx}}_{fin}$ (*resp.*, tx_{fin}) extends $\mathsf{tx}_{fin} \backslash \{\rho_{fin}\}$ with the partial redeemer $\hat{\rho}_{fin}$ (*resp.*, the full redeemer ρ_{fin}). Put $m = \mathsf{tx}_{fin} \backslash \{\rho_{fin}\}$. $\mathcal{O}_{\mathsf{CliRet}}$ records m in a queried set Q_{CliRet};

– $\mathcal{O}_{\mathsf{CliInitRet}}(\mathsf{pp}, sk_{\mathcal{C}}, \cdot)$. Similar to $\mathcal{O}_{\mathsf{CliRet}}$, on *valid* query $(R, \mathsf{tx}^n_{test}, \mathsf{tx}_{fin}\backslash\{\rho_{fin}\})$, it samples w and *honestly* executes $\mathsf{InitRet}$ to obtain $(C, \mathsf{aux}_{\mathsf{ret}}, \hat{\mathsf{tx}}_{fin})$, where $\hat{\mathsf{tx}}_{fin}$ extends $\mathsf{tx}_{fin}\backslash\{\rho_{fin}\}$ with the partial redeemer $\hat{\rho}_{fin}$. Put $m = \mathsf{tx}_{fin}\backslash\{\rho_{fin}\}$. $\mathcal{O}_{\mathsf{CliInitRet}}$ records $(\mathsf{aux}_{\mathsf{ret}}, \hat{\rho}_{fin})$ in its queried mapping set $Q_{\mathsf{CliInitRet}}$ under key m, *i.e.*, $(\mathsf{aux}_{\mathsf{ret}}, \hat{\rho}_{fin}) \in Q_{\mathsf{CliInitRet}}[m]$;
– $\mathcal{O}_{\mathsf{NewAux}}(\mathsf{pp}, \cdot)$. On query R, it samples a fresh w and executes $\mathsf{InitRet}$ on behalf of an *honest server*:

$$(C, \mathsf{aux}_{\mathsf{ret}}, \cdot) \leftarrow \mathsf{InitRet}(\cdot, \langle \cdot, \mathcal{S}(R, w)\rangle).$$

$\mathcal{O}_{\mathsf{NewAux}}$ returns $\mathsf{aux}_{\mathsf{ret}}$ to $\mathcal{A}$ and records $\mathsf{aux}_{\mathsf{ret}}$ in a queried set Q_{NewAux}.

Experiment $\mathsf{Expt}^{\text{adapt-cli-fair}}_{\mathsf{DSN},\mathcal{A},\mathcal{E}}(\mathsf{pp})$

$(D, R, \mathsf{tx}^n_{test}, C, \mathsf{aux}_{\mathsf{ret}}, \mathsf{tx}^*_{fin}) \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pp})$
Assert $(D \leftarrow \mathsf{Extract}(id, \tau_D, \mathsf{aux}, R)) \wedge \mathsf{tx}^n_{test} \in \mathcal{L}$
$b \leftarrow \mathsf{RetVrfy}(id, \tau_D, \mathsf{aux}, C, \mathsf{aux}_{\mathsf{ret}})$
$w_{\mathcal{E}} \leftarrow \mathcal{E}^{\mathcal{A}}(\mathsf{pp}, Q_{\mathsf{SimAux}})$
Return 1 if $b = 1 \wedge ((R_{\mathcal{E}}, \bot) \leftarrow \mathsf{RetExt}(C, w_{\mathcal{E}}))$
$\wedge\, (id, \tau_D, \mathsf{aux}, C, \mathsf{aux}_{\mathsf{ret}}) \notin Q_{\mathsf{SimAux}}$
$m^* \leftarrow \mathsf{tx}^*_{fin}\backslash\{\rho^*_{fin}\}$
Assert $\mathcal{O}_{\mathsf{txVrfy}}(\mathsf{tx}^*_{fin}) \wedge \nu^n_{test}(\rho^*_{fin}) \wedge (m^*, \rho^*_{fin}) \notin Q_{\mathsf{CliRet}}$
Return $\forall(\mathsf{aux}_{\mathsf{ret}}, \hat{\mathsf{tx}}_{fin}) \in Q_{\mathsf{CliInitRet}}[m^*]$ *s.t.* $\mathsf{aux}_{\mathsf{ret}} \notin Q_{\mathsf{NewAux}}$:
$((\cdot, \cdot, \bot) \leftarrow \mathsf{RetExt}(C, \mathsf{tx}^*_{fin}))$

Fig. 7: The experiment for adaptive client-fairness. The adversary $\mathcal{A}$ receives pp and is granted access to oracles $\mathcal{O} := \{\mathcal{O}_{\mathsf{CliDelPh}}, \mathcal{O}_{\mathsf{SimAux}}, \mathcal{O}_{\mathsf{CliRet}}, \mathcal{O}_{\mathsf{CliInitRet}}, \mathcal{O}_{\mathsf{NewAux}}\}$. $\mathcal{A}$ outputs (md, D, τ_D) of its choice and interacts with $\mathcal{O}_{\mathsf{CliDelPh}}$ to obtain (R, tx_{con}). The asserting verification ensures, through the extractability of R and contract continuity, that $\mathcal{A}$ executes the preservation phase honestly, *i.e.*, $\mathsf{tx}^n_{test} \in \mathcal{L}$. $\mathcal{A}$ wins if it outputs either: (1) a tuple $(C, \mathsf{aux}_{\mathsf{ret}})$ *s.t.* $\mathsf{RetVrfy}$ outputs 1, and there exists an extractor $\mathcal{E}^{\mathcal{A}}$ that outputs a witness key $w_{\mathcal{E}}$ which fails under RetExt; or (2) a valid forgery tx^*_{fin} that was never queried to $\mathcal{O}_{\mathsf{CliRet}}$ nor finalized from any partial transactions obtained from $\mathcal{O}_{\mathsf{CliInitRet}}$.

Definition 8 (Adaptive Client-Fairness). *A blockchain-aided DSN protocol has adaptive client-fairness if, for any PPT adversary $\mathcal{A}$ with access to oracles $\mathcal{O}_{\mathsf{CliDelPh}}, \mathcal{O}_{\mathsf{SimAux}}, \mathcal{O}_{\mathsf{CliRet}}, \mathcal{O}_{\mathsf{CliInitRet}}, \mathcal{O}_{\mathsf{NewAux}}$, there exists an extractor $\mathcal{E}^{\mathcal{A}}$, s.t. the following probability is $\mathsf{negl}(\lambda)$ for any $\lambda > 0$ and $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \cdot)$*

$$\Pr\left[\mathsf{Expt}^{\text{adapt-cli-fair}}_{\mathsf{DSN},\mathcal{A},\mathcal{E}}(\mathsf{pp}) = 1\right].$$

Server-fairness. To break server-fairness, a malicious client must[17] learn information about the replica R concealed under a witness key w prior to actually obtaining w. We model this as an indistinguishability game using a left-or-right oracle $\mathcal{O}_{\mathsf{RetLoR}}$. Additionally, we provide the adversary $\mathcal{A}$ with an oracle $\mathcal{O}_{\mathsf{SerPreRet}}$, which honestly executes the delegation and preservation phases on queried tuple (md, D, τ_D), returning the corresponding valid tx^n_{test}. The oracles are given below, and the experiment for server-fairness is provided in Figure 8.

- $\mathcal{O}_{\mathsf{RetLoR}}(\mathsf{pp}, w, \{(\mathsf{md}_i, D_i, \tau_{D_i})\}_{i\in\{0,1\}}; b)$ is a challenge oracle *w.r.t.* a bit $b \in \{0,1\}$. Let Q_{RetLoR} record the challenge inputs provided in the first query to $\mathcal{O}_{\mathsf{RetLoR}}$. If $Q_{\mathsf{RetLoR}} \neq \emptyset \wedge Q_{\mathsf{RetLoR}} \neq \{(\mathsf{md}_i, D_i, \tau_{D_i})\}_{i\in\{0,1\}}$, $\mathcal{O}_{\mathsf{RetLoR}}$ returns $\perp$. Otherwise, it *honestly* executes the delegation and preservation phase on $(\mathsf{md}_b, D_b, \tau_{D_b})$ to obtain $(\mathsf{tx}^n_{test,b}, R_b)$. It then performs $\mathsf{InitRet}$ with $\mathcal{A}$ impersonating a malicious client, *i.e.*,
$$(C_b, \mathsf{aux}_{\mathsf{ret},b}, \cdot) \leftarrow \mathsf{InitRet}(\mathsf{tx}^n_{test,b}, \langle \mathcal{A}(sk_{\mathcal{A}}), \mathcal{S}(R_b, w)\rangle) \ s.t.$$
$\mathsf{RetVrfy}(id_b, \tau_{D_b}, \mathsf{aux}_b, C_b, \mathsf{aux}_{\mathsf{ret},b}) = 1$ and returns $(\mathsf{tx}^n_{test,b}, C_b, \mathsf{aux}_{\mathsf{ret},b})$;
- $\mathcal{O}_{\mathsf{SerPreRet}}(\mathsf{pp}, sk_{\mathcal{S}}, \cdot)$. On query (md, D, τ_D), it returns $\perp$ if $\mathsf{TagVrfy}(D, \tau_D) = 0$. Otherwise, $\mathcal{O}_{\mathsf{SerPreRet}}$ performs the delegation and preservation phase with $\mathcal{A}$ impersonating a client, returns $\mathsf{tx}^n_{test} \in \mathcal{L}$ to $\mathcal{A}$, and records (md, D, τ_D) in the queried set $Q_{\mathsf{SerPreRet}}$.

Experiment $\mathsf{Expt}^{\text{ser-fair}}_{\mathsf{DSN},\mathcal{A}}(\mathsf{pp}, w)$

$(\{(\mathsf{md}_i, D_i, \tau_{D_i})\}_{i\in\{0,1\}}, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp})$ *s.t.* $\mathsf{TagVrfy}(D_i, \tau_{D_i}) = 1$

$b \xleftarrow{\$} \{0,1\};\ b' \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pp}, \mathsf{st})$

Return 1 if $b' = b \wedge \{(\mathsf{md}_i, D_i, \tau_{D_i})\}_{i\in\{0,1\}} \notin Q_{\mathsf{SerPreRet}}$

Fig. 8: The experiment for server-fairness. The adversary $\mathcal{A}$ receives pp and outputs $\{(\mathsf{md}_i, D_i, \tau_{D_i})\}_{i\in\{0,1\}}$ of its choice. $\mathcal{A}$ is granted access to oracles $\mathcal{O} := \{\mathcal{O}_{\mathsf{RetLoR}}, \mathcal{O}_{\mathsf{SerPreRet}}\}$, where $\mathcal{O}_{\mathsf{RetLoR}}$ answers only once *w.r.t.* the challenge bit b, using the freshly assigned w. $\mathcal{A}$ wins if it correctly guesses $b' = b$, and neither tuple in $\{(\mathsf{md}_i, D_i, \tau_{D_i})\}_{i\in\{0,1\}}$ has been previously queried to $\mathcal{O}_{\mathsf{SerPreRet}}$.

Definition 9 (Server-Fairness). *A blockchain-aided DSN has server-fairness if, for any PPT adversary $\mathcal{A}$ with access to oracles $\mathcal{O}_{\mathsf{RetLoR}}, \mathcal{O}_{\mathsf{SerPreRet}}$, then*
$$\left|\Pr\left[\mathsf{Expt}^{\text{ser-fair}}_{\mathsf{DSN},\mathcal{A}}(\mathsf{pp}, w) = 1\right] - \frac{1}{2}\right|$$
is $\mathsf{negl}(\lambda)$ *for any* $\lambda > 0$, $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \cdot)$ *and any* w.

[17] One may consider an alternative attack where a malicious client outputs a valid $\hat{\mathsf{tx}}_{fin}$ and deceives an honest server into finalizing it with the corresponding w, yet the resulting tx_{fin} is invalid. However, this attack fails in our model since both $\mathsf{Finalize}$ and the validity check of tx_{fin} can be performed solely by the server. A rational server will refuse to reveal w if she cannot produce a valid tx_{fin}.

4 Generic Construction and Security Analysis

This section begins by reviewing necessary building blocks for our generic DSN construction and, rather than giving concrete instantiations, state the properties they must satisfy. We then present the construction and its security analysis.

4.1 Building Blocks

What underpins all phases is a space-time PoRep variant (Remark 1), *i.e.*, $\mathsf{PoRep} = (\mathsf{Setup}, \mathsf{Preproc}, \mathsf{TagVrfy}, \mathsf{Replicate}, \mathsf{Extract}, \mathsf{Poll}, \mathsf{Prove}, \mathsf{Vrfy})$, which satisfies correctness and replication soundness. We also require that the implicit Proof of Retrievable Commitment (PoRC) of PoRep is binding (see Appendix B).

To realize fair data retrieval, we adopt an encryption-then-adaptor-signature approach [41]. Specifically, we employ a secure symmetric encryption scheme $\mathsf{SKE} = (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$. For the adaptor signature functionality, we *inherently* employ the generic construction from [13], which extends a secure signature scheme $\mathsf{SIG} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ using a relation primitive $\mathsf{R} = (\mathsf{Gen}, \mathsf{Vrfy})$ associated with an NP relation $\mathcal{R}$ that satisfies hardness and has unique witness.

Finally, we define two relations and their respective non-interactive argument protocols. First, let $\mathcal{R}_{\mathsf{PoRep}}$ be the relation corresponding to $\mathsf{PoRep.Vrfy}$, *i.e.*, for any instance $(id, \tau_D, \mathsf{aux}, \mathsf{ch})$ and witness $(R, \pi_{\mathsf{PoRep}})$ pair, that verifies $\mathcal{R}_{\mathsf{PoRep}}((id, \tau_D, \mathsf{aux}, \mathsf{ch}), (R, \pi_{\mathsf{PoRep}})) = 1$, if and only if,

$$\begin{gathered}\pi_{\mathsf{PoRep}} \leftarrow \mathsf{PoRep.Prove}(id, R, \mathsf{aux}, \mathsf{ch}) \\ \wedge \mathsf{PoRep.Vrfy}(id, \tau_D, \mathsf{aux}, \mathsf{ch}, \pi_{\mathsf{PoRep}}) = 1.\end{gathered}$$

We also support the overloaded notations for $\mathsf{PoRep.Replicate}$ and the initial verification, where explicitly $\mathsf{aux} = (\tau_R, \mathsf{ch}_{\mathsf{aux}}, \pi_{\mathsf{aux}})$. In this case, the relation verifies $\mathcal{R}_{\mathsf{PoRep}}((id, \tau_D, \tau_R, \mathsf{ch}_{\mathsf{aux}}), (R, \pi_{\mathsf{aux}})) = 1$, if and only if,

$$\begin{gathered}(R, \tau_R, \pi_{\mathsf{aux}}) \leftarrow \mathsf{PoRep.Replicate}(id, D, \tau_D, \mathsf{ch}_{\mathsf{aux}}) \\ \wedge \mathsf{PoRep.Vrfy}(id, \tau_D, \mathsf{aux}) = 1.\end{gathered}$$

Second, the relation used during retrieval, denoted by $\mathcal{R}_{\mathsf{InitRet}}$, is defined as follows. For any instance $(id, \tau_D, \mathsf{aux}, C, x_K)$ and witness (w_K, R), parse $\mathsf{aux} = (\tau_R, \mathsf{ch}_{\mathsf{aux}}, \pi_{\mathsf{aux}})$, $\mathcal{R}_{\mathsf{InitRet}}((id, \tau_D, \mathsf{aux}, C, x_K), (w_K, R)) = 1$, if and only if,

$$\begin{gathered}\mathcal{R}_{\mathsf{PoRep}}((id, \tau_D, \tau_R, \mathsf{ch}_{\mathsf{aux}}), (R, \pi_{\mathsf{aux}})) = 1 \wedge \\ \mathsf{R.Vrfy}(x_K, w_K) = 1 \wedge C \leftarrow \mathsf{SKE.Enc}(w_K, R),\end{gathered}$$

where we emphasize that the same replica R must be used in proving the relation $\mathcal{R}_{\mathsf{PoRep}}$ and in the encryption $\mathsf{SKE.Enc}$.

We instantiate $\mathcal{R}_{\mathsf{PoRep}}$ with a zkSNARK protocol $\Pi_{\mathcal{R}}^{\mathsf{PoRep}}$, and $\mathcal{R}_{\mathsf{InitRet}}$ with an NIZKPoK protocol[18] $\Pi_{\mathcal{R}}^{\mathsf{InitRet}}$.

[18] $\Pi_{\mathcal{R}}^{\mathsf{InitRet}}$ can be interactive if desired. However, to maintain a unified abstraction, we refer to it under the non-interactive argument framework defined in Appendix A.1

4.2 Protocol Construction and Transaction Specifications

This section details the construction of each protocol phase, structured around the PoRep backbone, and specifies the corresponding transactions.

Preprocessing phase and preparations. The phase is composed by the procedures $(\mathsf{Setup}, \mathsf{TSetup}, \mathsf{Preproc})$, and they are performed as follows.

- $\mathsf{Setup}(1^\lambda, t)$: Run $\mathsf{PoRep.pp} \leftarrow \mathsf{PoRep.Setup}(1^\lambda, t)$;

$$(\mathsf{crs}_{\mathsf{PoRep}}, \mathsf{td}_{\mathsf{PoRep}}) \leftarrow \Pi_{\mathcal{R}}^{\mathsf{PoRep}}.\mathsf{Gen}(1^\lambda, \mathcal{R}_{\mathsf{PoRep}});$$
$$\text{and } (\mathsf{crs}_{\mathsf{InitRet}}, \mathsf{td}_{\mathsf{InitRet}}) \leftarrow \Pi_{\mathcal{R}}^{\mathsf{InitRet}}.\mathsf{Gen}(1^\lambda, \mathcal{R}_{\mathsf{InitRet}}).$$

  Set $\mathsf{pp} = (\mathsf{PoRep.pp}, \mathsf{crs}_{\mathsf{PoRep}}, \mathsf{crs}_{\mathsf{InitRet}}, pk_{\mathcal{B}}, T(\lambda, t))$ and output it. As before, we omit pp when the context is clear;
- $\mathsf{TSetup}(1^\lambda, t)$: In addition to pp above, output also $\mathsf{td} = (\mathsf{td}_{\mathsf{PoRep}}, \mathsf{td}_{\mathsf{InitRet}})$; and
- $\mathsf{Preproc}(ek, \tilde{D})$: Run $\mathsf{PoRep.Preproc}$, which is designed to support secret-keyed data preprocessing, and output (D, τ_D).

Preparations. For the parties $\mathcal{C}$ and $\mathcal{S}$ participating in the following phases (*i.e.*, delegation, preservation, and retrieval), we prepare their respective inputs: key pairs, and transactions for payment and collateral.

Concretely, $\mathcal{C}$ (*resp.*, $\mathcal{S}$) holds $(sk_{\mathcal{C}}, pk_{\mathcal{C}}) \leftarrow \mathsf{SIG.KGen}(1^\lambda)$ (*resp.*, $(sk_{\mathcal{S}}, pk_{\mathcal{S}}) \leftarrow \mathsf{SIG.KGen}(1^\lambda)$), as well as a payment out_{pay} (*resp.*, collateral output out_{col}). We further specify the validators ν_{pay}, ν_{col} as follows:

$$\nu_{pay} = \textsc{Script}(\rho_{con}^{pay}, \mathsf{tx}_{con})\{\textsc{Return 1 If}: \ \mathsf{SIG.Vrfy}(pk_{\mathcal{C}}, \tilde{\mathsf{tx}}_{con}, \rho_{con}^{pay})\}, \text{ and}$$
$$\nu_{col} = \textsc{Script}(\rho_{con}^{col}, \mathsf{tx}_{con})\{\textsc{Return 1 If}: \ \mathsf{SIG.Vrfy}(pk_{\mathcal{S}}, \tilde{\mathsf{tx}}_{con}, \rho_{con}^{col})\},$$

where $\tilde{\mathsf{tx}}_{con}$ denotes the base transaction (cf. Section 2.1) of tx_{con}, as specified in OnDel of the delegation phase below. Furthermore, we assume that $\mathcal{C}$ and $\mathcal{S}$ agree on a contract metadata $\mathsf{md} = (id, n, V)$.

Delegation phase. The phase is composed by the five procedures in the tuple $(\mathsf{OffDel}, \mathsf{TagVrfy}, \mathsf{DelVrfy}, \mathsf{Extract}, \mathsf{OnDel})$, and they are performed as follows.

- $\mathsf{OffDel}(\mathsf{md}, \langle \mathcal{C}(D, \tau_D), \mathcal{S} \rangle)$: Obtain $id \in \mathsf{md}$, the off-chain delegation subprotocol between $\mathcal{C}$ and $\mathcal{S}$ is illustrated by Figure 9.
- $\mathsf{TagVrfy}, \mathsf{Extract}$: Run $\mathsf{PoRep.TagVrfy}$ and $\mathsf{PoRep.Extract}$, respectively;
- $\mathsf{DelVrfy}(id, \tau_D, \mathsf{aux})$: Parse $\mathsf{aux} = (\tau_R, \mathsf{ch}_{\mathsf{aux}}, \pi)$ and output

$$\Pi_{\mathcal{R}}^{\mathsf{PoRep}}.\mathsf{Vrfy}(\mathsf{crs}_{\mathsf{PoRep}}, (id, \tau_D, \tau_R, \mathsf{ch}_{\mathsf{aux}}), \pi).$$

- $\mathsf{OnDel}(m_{con}, \mathsf{tx}_{pay}, \mathsf{tx}_{col}, \langle \mathcal{C}(sk_{\mathcal{C}}), \mathcal{S}(sk_{\mathcal{S}}) \rangle)$: Obtain vi_{con} from $V \in \mathsf{md}$, the on-chain delegation subprotocol between $\mathcal{C}$ and $\mathcal{S}$ is illustrated by Figure 10.

OffDel Subprotocol

- $\mathcal{C}$ sends (D, τ_D) to $\mathcal{S}$;
- $\mathcal{S}$ runs $b \leftarrow \mathsf{TagVrfy}(D, \tau_D)$ and aborts with $\perp$, if b=0. Otherwise, she runs $(R, \tau_R, \pi_{\mathsf{aux}}) \leftarrow \mathsf{PoRep.Replicate}(id, D, \tau_D, \mathsf{ch}_{\mathsf{aux}})$, compresses the proof with $\pi \leftarrow \mathit{\Pi}_{\mathcal{R}}^{\mathsf{PoRep}}.\mathsf{Prove}(\mathsf{crs}_{\mathsf{PoRep}}, (id, \tau_D, \tau_R, \mathsf{ch}_{\mathsf{aux}}), (R, \pi_{\mathsf{aux}}))$, sets $\mathsf{aux} = (\tau_R, \mathsf{ch}_{\mathsf{aux}}, \pi)$, and sends aux to $\mathcal{C}$;
- $\mathcal{C}$ runs $b \leftarrow \mathsf{DelVrfy}(id, \tau_D, \mathsf{aux})$ and aborts with $\perp$ if $b = 0$;
- Set $m_{con} = (\mathsf{md}, \tau_D, \mathsf{aux})$ and output $\langle \mathcal{C}(m_{con}), \mathcal{S}(R, m_{con}) \rangle$ if neither party aborts, or output $\perp$ otherwise.

Fig. 9: TagVrfy and DelVrfy verify, respectively, the data tag and delegation, and they are both specified below.

OnDel Subprotocol

- Abort with $\perp$ if $m_{con} = \perp \vee \mathsf{tx}_{pay} \notin \mathcal{L} \vee \ \mathsf{tx}_{col} \notin \mathcal{L}$;
- Prepare the base transaction $\tilde{\mathsf{tx}}_{con} = (\tilde{\mathsf{in}}_{con}^{pay}, \tilde{\mathsf{in}}_{con}^{col}, \mathsf{out}_{con}^{pay}, \mathsf{out}_{con}^{pay}, vi_{con})$, where $\tilde{\mathsf{in}}_{con}^{pay} = (\mathsf{out}_{pay}, \perp), \tilde{\mathsf{in}}_{con}^{col} = (\mathsf{out}_{col}, \perp)$ and $\mathsf{out}_{con}^{pay} = (\nu_{con}^{pay}, p, m_{con}), \mathsf{out}_{con}^{col} = (\nu_{con}^{col}, c, m_{con})$ *s.t.*

$$\begin{aligned}
\nu_{con}^{col} = &\textsc{Script}(\rho_{test}^1, \mathsf{tx}_{test}^1)\{ \\
&\quad \textsc{Return 1 If}: \ \delta_{test}^1 = m_{con} \wedge \\
&\qquad \mathsf{TestVrfy}^1(id, \tau_D, \mathsf{aux}, \mathsf{ch}^1, \mathsf{tx}_{con} \backslash \{\nu_{con}^{col}\}, \pi^1) \wedge \\
&\qquad \mathsf{SIG.Vrfy}(pk_{\mathcal{S}}, \tilde{\mathsf{tx}}_{test}^1, \sigma^1) \\
&\quad \textsc{Else}: \ \textsc{Lock } c \textsc{ under } pk_{\mathcal{B}}\},
\end{aligned}$$

  where ρ_{test}^1 is parsed as $(\mathsf{ch}^1, \pi^1, \sigma^1)$ and $\tilde{\mathsf{tx}}_{test}^1 = \mathsf{tx}_{test}^1 \backslash \{\sigma^1\}$, *i.e.*, a base transaction with signature unattached (cf. Section 2.1).
- $\mathcal{C}$ and $\mathcal{S}$ both sign the base transaction, *i.e.*, $\sigma_{con}^{pay} \leftarrow \mathsf{SIG.Sign}(sk_{\mathcal{C}}, \tilde{\mathsf{tx}}_{con})$ and $\sigma_{con}^{col} \leftarrow \mathsf{SIG.Sign}(sk_{\mathcal{S}}, \tilde{\mathsf{tx}}_{con})$, set $\rho_{con}^{pay} = \sigma_{con}^{pay}$ and $\rho_{con}^{col} = \sigma_{con}^{col}$, and output the completed contract transaction tx_{con}.

Fig. 10: The generation of ρ_{test}^1, and $\mathsf{TestVrfy}^1$ are specified below *w.r.t.* index i.

Remark 5 (On the signing process). $\mathcal{C}$ and $\mathcal{S}$ jointly sign the base contract transaction $\tilde{\mathsf{tx}}_{con}$ in OnDel, which can be efficiently realized using multi-signature schemes [5]. We further require $\mathcal{S}$ to sign first. This is because ch^1 is deterministically derived via PoRep.Poll from the full tx_{con}. If $\mathcal{S}$ receives a partially signed $\tilde{\mathsf{tx}}_{con} \cup \{\sigma_{con}^{pay}\}$ and appends her own signature to obtain the full tx_{con}, she can derive ch^1 prior to publishing tx_{con}. Thus giving $\mathcal{S}$ additional time to generate the proof and violating the testing interval constraint enforced by $T \in \mathsf{pp}$.

Preservation phase. As before, we put $\mathsf{tx}^0_{test} = \mathsf{tx}_{con}$. The algorithms Test^i and $\mathsf{TestVrfy}^i$ for index $i \in [n]$ are performed as follows. In particular, for $i = n$, the specifications of ν^n_{test} and $\mathsf{tx}^{n+1}_{test} \coloneqq \mathsf{tx}_{fin}$ are deferred to the retrieval phase.

– $\mathsf{Test}(\mathsf{tx}^{i-1}_{test}, R)$: Abort with $\perp$ if $\mathsf{tx}^{i-1}_{test} \notin \mathcal{L}$. Obtain vi_{test} from $V{\in}\mathsf{md}$;
  - Obtain $(id, \tau_D, \mathsf{aux})$ from $m_{con} \in \delta^{i-1}_{test}$ (*i.e.*, the datum), and derive challenge ch^i by using $\mathsf{tx}^{i-1}_{test}\backslash\{\nu^{i-1}_{test}\}$ (*i.e.*, ν^{i-1}_{test} is removed to avoid circular dependency) as the public seed, *i.e.*, $\mathsf{ch}^i \leftarrow \mathsf{PoRep.Poll}(\mathsf{aux}, \mathsf{tx}^{i-1}_{test}\backslash\{\nu^{i-1}_{test}\})$. Run $\pi_{\mathsf{PoRep}} \leftarrow \mathsf{PoRep.Prove}(id, R, \mathsf{aux}, \mathsf{ch}^i)$ and compress the proof with $\pi^i \leftarrow \mathit{\Pi}^{\mathsf{PoRep}}_{\mathcal{R}}.\mathsf{Prove}(\mathsf{crs}_{\mathsf{PoRep}}, (id, \tau_D, \mathsf{aux}, \mathsf{ch}^i), (R, \pi_{\mathsf{PoRep}}))$;
  - Prepare the base transaction $\tilde{\mathsf{tx}}^i_{test} = (\tilde{\mathsf{in}}^i_{test}, \mathsf{out}^i_{test}, vi_{test})$, where $\tilde{\mathsf{in}}^i_{test} = (\mathsf{out}^{i-1}_{test}, (\mathsf{ch}^i, \pi^i, \perp))$ and $\mathsf{out}^i_{test} = (\nu^i_{test}, c, m_{con})$, *s.t.*

$$\begin{aligned}\nu^i_{test} =& \textsc{Script}(\rho^{i+1}_{test}, \mathsf{tx}^{i+1}_{test})\{\\ &\textsc{Return 1 If}: \ \delta^{i+1}_{test} = m_{con} \wedge\\ &\quad \mathsf{TestVrfy}^{i+1}(id, \tau_D, \mathsf{aux}, \mathsf{ch}^{i+1}, \mathsf{tx}^i_{test}\backslash\{\nu^i_{test}\}, \pi^{i+1}) \wedge\\ &\quad \mathsf{SIG.Vrfy}(pk_{\mathcal{S}}, \tilde{\mathsf{tx}}^{i+1}_{test}, \sigma^{i+1})\\ &\textsc{Else}: \ \textsc{Lock } c \textsc{ under } pk_{\mathcal{B}}\},\end{aligned}$$

    where ρ^{i+1}_{test} is parsed as $(\mathsf{ch}^{i+1}, \pi^{i+1}, \sigma^{i+1})$ and $\tilde{\mathsf{tx}}^{i+1}_{test} = \mathsf{tx}^{i+1}_{test}\backslash\{\sigma^{i+1}\}$;
  - Sign the base transaction with $\sigma^i \leftarrow \mathsf{SIG.Sign}(sk_{\mathcal{S}}, \tilde{\mathsf{tx}}^i_{test})$, set $\rho^i_{test} = (\mathsf{ch}^i, \pi^i, \sigma^i)$, and output the completed test transaction tx^i_{test};
– $\mathsf{TestVrfy}^i(id, \tau_D, \mathsf{aux}, \mathsf{ch}^i, \mathsf{seed}^i, \pi^i)$: Parse $\mathsf{seed}^i = \mathsf{tx}^{i-1}_{test}\backslash\{\nu^{i-1}_{test}\}$, compute $\mathsf{ch}'^i \leftarrow \mathsf{PoRep.Poll}(\mathsf{aux}, \mathsf{seed}^i)$, and output

$$\mathsf{ch}'^i = \mathsf{ch}^i \wedge \mathit{\Pi}^{\mathsf{PoRep}}_{\mathcal{R}}.\mathsf{Vrfy}(\mathsf{crs}_{\mathsf{PoRep}}, (id, \tau_D, \mathsf{aux}, \mathsf{ch}^i), \pi^i).$$

Retrieval phase. Here, we first specify $\nu^n_{test} \in \mathsf{tx}^n_{test}$ *w.r.t.* tx_{fin} as follows

$$\begin{aligned}\nu^n_{test} =& \textsc{Script}(\rho_{fin}, \mathsf{tx}_{fin})\{\\ &\textsc{Return 1 If}: \ \delta_{fin} = m_{con} \wedge\\ &\quad \mathsf{SIG.Vrfy}(pk_{\mathcal{C}}, \tilde{\mathsf{tx}}_{fin}, \hat{\rho}_{fin}) \wedge \mathsf{R.Vrfy}(x_K, w_K)\\ &\textsc{Else}: \ \textsc{Lock } c \textsc{ under } pk_{\mathcal{B}}\},\end{aligned}$$

where $\hat{\rho}_{fin}$ is parsed as $(x_K, \hat{\sigma})$ (hence, $\tilde{\mathsf{tx}}_{fin} = \hat{\mathsf{tx}}_{fin}\backslash\{\hat{\sigma}\}$), and ρ_{fin} as $(x_K, \hat{\sigma}, w_K)$ (*i.e.*, $(\hat{\rho}_{fin}, w_K)$). Recall the partial transaction validation, $\nu^n_{test}(\hat{\rho}_{fin})$ performs only the signature verification $\mathsf{SIG.Vrfy}(pk_{\mathcal{C}}, \tilde{\mathsf{tx}}_{fin}, \hat{\rho}_{fin})$. The generation of these values is given within the retrieval phase tuple $(\mathsf{InitRet}, \mathsf{RetVrfy}, \mathsf{Finalize}, \mathsf{RetExt})$.

– $\mathsf{InitRet}(\mathsf{tx}^n_{test}, \langle \mathcal{C}(sk_{\mathcal{C}}), \mathcal{S}(R, \cdot)\rangle)$: Obtain vi_{fin} from $V \in \mathsf{md}$. The initial retrieval subprotocol between $\mathcal{C}$ and $\mathcal{S}$ is illustrated by Figure 11;
– $\mathsf{RetVrfy}(id, \tau_D, \mathsf{aux}, C, \mathsf{aux}_{\mathsf{ret}})$: Parse $\mathsf{aux}_{\mathsf{ret}} = (x_K, \pi_{\mathsf{ret}})$ and output

$$\mathit{\Pi}^{\mathsf{InitRet}}_{\mathcal{R}}.\mathsf{Vrfy}(\mathsf{crs}_{\mathsf{InitRet}}, (id, \tau_D, \mathsf{aux}, C, x_K), \pi_{\mathsf{ret}});$$

InitRet Subprotocol

- Abort with $\perp$ if $\mathsf{tx}^n_{test} \notin \mathcal{L}$;
- $\mathcal{S}$ runs $(x_K, w_K) \leftarrow \mathsf{R.Gen}(1^\lambda)$ and computes $C \leftarrow \mathsf{SKE.Enc}(w_K, R)$. She runs $\pi_{\mathsf{ret}} \leftarrow \Pi^{\mathsf{InitRet}}_{\mathcal{R}}.\mathsf{Prove}(\mathsf{crs}_{\mathsf{InitRet}}, (id, \tau_D, \mathsf{aux}, C, x_K), (w_K, R))$ and sends C and $\mathsf{aux}_{\mathsf{ret}} = (x_K, \pi_{\mathsf{ret}})$ to $\mathcal{C}$;
- $\mathcal{C}$ runs $b \leftarrow \mathsf{RetVrfy}(id, \tau_D, \mathsf{aux}, C, \mathsf{aux}_{\mathsf{ret}})$ and aborts with $\perp$ if $b = 0$;
- $\mathcal{C}$ prepares the base transaction $\tilde{\mathsf{tx}}_{fin} = (\tilde{\mathsf{in}}_{fin}, \mathsf{out}_{fin}, vi_{fin})$, where $\tilde{\mathsf{in}}_{fin} = (\mathsf{out}^n_{test}, (x_K, \perp))$ and $\mathsf{out}_{fin} = (\nu_{fin}, c, m_{con})$. She signs with $\hat{\sigma} \leftarrow \mathsf{SIG.Sign}(sk_{\mathcal{C}}, \tilde{\mathsf{tx}}_{fin})$, completes $\hat{\rho}_{fin} = (x_K, \hat{\sigma})$, and outputs the partial transaction $\hat{\mathsf{tx}}_{fin}$;

Fig. 11: RetVrfy is one of the procedures performed during the Retrieval phase.

– $\mathsf{Finalize}(w_K, \hat{\mathsf{tx}}_{fin})$: Parse $\hat{\rho}_{fin} = (x_K, \hat{\sigma})$ and $\tilde{\mathsf{tx}}_{fin} = \hat{\mathsf{tx}}_{fin} \backslash \{\hat{\sigma}\}$. Run $b \leftarrow \mathcal{O}_{\mathsf{txVrfy}}(\hat{\mathsf{tx}}_{fin}) \wedge \mathsf{SIG.Vrfy}(pk_{\mathcal{C}}, \tilde{\mathsf{tx}}_{fin}, \hat{\sigma})$ and abort with $\perp$ if $b = 0$. Set $\rho_{fin} = (\hat{\rho}_{fin}, w_K)$ and output the full finalization transaction tx_{fin}; and
– $\mathsf{RetExt}(C, \mathsf{tx}_{fin})$: Abort with $\perp$ if $\mathsf{tx}_{fin} \notin \mathcal{L}$. Parse $\rho_{fin} = (\hat{\rho}_{fin}, w_K)$. Run $R \leftarrow \mathsf{SKE.Dec}(w_K, C)$ and $D \leftarrow \mathsf{Extract}(id, \tau_D, \mathsf{aux}, R)$. Output $\perp$ if any call returns $\perp$; otherwise, output (w_K, R, D).

Our construction realizes the verifiable encryption with commitment keys (VECK) functionality [41], which enables fair data exchange using generic cryptographic primitives. In contrast, concrete instantiations are provided in *loc. cit.*, with ElGamal-based and Paillier-based designs tailored for KZG commitments, offering better efficiency. We conjecture that their approach could directly replace our generic construction with minimal modifications: by substituting the KZG commitment with a simulation-extractable variant [34], the resulting instantiation is expected to preserve our adaptive client-fairness property while improving performance. A full integration and formal security analysis of such an instantiation is left for future work.

Remark 6 (Unspecified validators). One may notice that the validators $\nu^{pay}_{con} \in \mathsf{tx}_{con}$ and $\nu_{fin} \in \mathsf{tx}_{fin}$ are not specified in the given construction. As out^{pay}_{con} transfers payment to $\mathcal{S}$ and out_{fin} refunds the collateral to $\mathcal{S}$, both validators can be straightforwardly implemented with signature verification scripts under $(sk_{\mathcal{S}}, pk_{\mathcal{S}})$. We intentionally omit their full specification to avoid introducing unnecessary redeeming transactions.

4.3 Security Analysis

We formally state the security guarantees of our DSN protocol as follows.

Theorem 1. *Given an EUTxO-based blockchain protocol $\Pi^{\mathcal{O}_{\mathsf{txVrfy}}}_{\mathcal{L},k,u}$ that satisfies k-persistence and (k,u)-liveness, the DSN protocol $\mathsf{DSN}_{\mathcal{L}}$ constructed in Section 4.2 satisfies the following properties.*

– *Correctness (Definition 4);*
– *Delegation binding (Definition 5) if the underlying* PoRep *is a binding PoRC (Appendix C);*
– *Contract unforgeability (Definition 6) if* SIG *satisfies EUF-CMA ;*
– *Adaptive replication soundness (Definition 7) if* PoRep *is replication sound, and* $\Pi_{\mathcal{R}}^{\mathsf{PoRep}}$ *is statistical zero-knowledge, and satisfies preprocessing succinctness and simulation-extractable knowledge soundness;*
– *Adaptive client-fairness (Definition 8) if* SIG *satisfies* SUF-CMA, R *is hard and has unique witness, and* $\Pi_{\mathcal{R}}^{\mathsf{InitRet}}$ *is statistical zero-knowledge, and satisfies simulation-extractable knowledge soundness;*
– *Server-fairness (Definition 9) if* SKE *is IND-CPA,* R *is hard, and* $\Pi_{\mathcal{R}}^{\mathsf{InitRet}}$ *is* at least *statistical zero-knowledge.*

Proof. We provide a brief proof here. *Correctness* follows directly from the correctness of cryptographic building blocks and the security of the EUTxO-based blockchain protocol. As for *delegation binding*, our constructed DSN protocol is identical to the implicit PoRC scheme of the underlying PoRep, and hence, the property follows directly from the binding of the PoRC (Definition 26). Similarly, *contract unforgeability* follows by invoking the unforgeability of SIG (Definition 13) twice, one for each side.

Adaptive replication soundness. Let Game_0 be the original experiment given in Figure 6. We define a single subsequent Game_1, in which the challenger, whenever required to produce a proof in $\mathcal{O}_{\mathsf{SimTest}}$, invokes the simulator of $\Pi_{\mathcal{R}}^{\mathsf{PoRep}}$ (Definition 18). By zero-knowledge, the view of any PPT adversary $\mathcal{A}$ remains.

Now, for every proof that verifies on-chain, *i.e.*, when $\mathsf{tx}_{test}^{i} \in \mathcal{L}$ and contains π^i, the adaptive replication soundness extractor $\mathcal{E}$ can invoke the simulation-extractability extractor (Definition 20), which in turn, calls the implicit PoRC extractor (Definition 27). Both procedures run in polynomial time, and $\mathcal{E}$ outputs $(R^{i-1}, \pi^i_{\mathsf{PoRep}})$. By simulation-extractability and PoRC soundness, the extraction process fails with negligible probability, and the output satisfies that π^i_{PoRep} is valid *w.r.t.* R^{i-1}. Again, by PoRC soundness, R^{i-1} is extractable, *i.e.*, $(D \leftarrow \mathsf{Extract}(id, \tau_D, \mathsf{aux}, R^{i-1})) = 1$.

Next, we consider the remaining bad events specified in the definition. They are: (1) for some $j < i$, the extracted replica size shrinks more than a fraction of ϵ_1, *i.e.*, $|R^j| < (1-\epsilon_1)|R^{j-1}|$; or (2) in iteration j, the adversary's internal state is $|\mathsf{st}^j| < (1-\epsilon_2)|R^j|$.

Both cases are ruled out by the incompressibility of extractable replicas (Definition 25), which is implied by the implicit PoSpace of PoRep. Case (1) follows directly from the incompressibility as both $|R^j|$ and $|R^{j-1}|$ are extractable. For case (2), suppose $|\mathsf{st}^j| < (1-\epsilon_2)|R^j|$ occurs in iteration j, and π^{j+1} is valid. Then, the total space to generate π^{j+1} is at most $|\mathsf{st}^j| + \sum_{l\in[j]} |\pi^l|$ (recall that the adversary is only allowed to leverage the proof space in transactions). By the succinctness of proofs (Definition 19), *i.e.*, the size of each proof is polynomial of λ and independent of replica size, there exists ϵ *s.t.* $|\mathsf{st}^j| + \sum_{l\in[j]} |\pi^l| < (1-\epsilon)|R^j|$, which violates the soundness of the implicit PoSpace.

Adaptive client-fairness. The experiment defines two disjoint winning conditions. For the first, similar to the simulation-extractability argument in the proof of adaptive replication soundness, we replace any query to $\mathcal{O}_{\mathsf{SimAux}}$ with the simulator of $\Pi_{\mathcal{R}}^{\mathsf{InitRet}}$, which gives an PPT adversary no advantage. Then, by simulation-extractability, the extractor outputs $w_{\mathcal{E}}$ with probability $1 - \mathsf{negl}(\lambda)$ *s.t.* $(R, D) \leftarrow \mathsf{RetExt}(C, w_{\mathcal{E}})$.

For the seconding condition, the adversary must output a valid adaptor signature whose witness cannot be extracted, which contradicts the strong full extractability of adaptor signature schemes. Given SIG is SUF-CMA secure and R is associated with a hard NP relation that has unique witness, then by Lemma 1, the construction based on SIG and R indeed satisfies strong full extractability. A notable technique in defining our $\mathcal{O}_{\mathsf{CliRet}}$, which ensures the above argument holds, is to leverage the canonical signing method, as described in the proof of Lemma 1 (given in [13, Theorem 3.4]).

Server-fairness. Whenever the challenger runs $\mathsf{InitRet}$ in $\mathcal{O}_{\mathsf{RetLoR}}$, it uses the simulator of $\Pi_{\mathcal{R}}^{\mathsf{InitRet}}$ to generate $\mathsf{aux}_{\mathsf{ret}} = (x_K, \pi_{\mathsf{Sim}})$ instead of producing a real proof. The hardness of R ensures that the adversary cannot recover w_K from x_K when w_K is freshly sampled for each $\mathsf{InitRet}$. Moreover, the *statistical* zero-knowledge of $\Pi_{\mathcal{R}}^{\mathsf{InitRet}}$ guarantees the simulated proof π_{Sim} is indistinguishable from a real one, even after the game concludes. The remaining challenge is to distinguish between C_0 and C_1 given C_b *w.r.t.* the challenge bit $b \in \{0, 1\}$. This directly reduces to the IND-CPA game of SKE (Definition 11). □

5 Implementation and Experimental Results

To further motivate our proposed security model, we conduct an experiment simulating the randomness-tampering attack described in Section 1.1 against a toy DSN-style protocol. The code is available in the anonymous repository[19].

Setup. The common input is a dummy file that consists of 100 blocks, each of size 512 bytes, and is committed via a Merkle tree. In each epoch, defined as one round of challenge-response interaction, a challenge targets multiple randomly selected block indices. We assume that an honest prover retains the full data at all times, whereas an attacker minimizes its local storage by offloading retrievable information to an external state that simulates the underlying blockchain in DSN protocols.

We tested four prover strategies. The first is an honest baseline. The second, a plain proof attacker, targets protocols that accept plain Merkle proofs. After answering a challenge, such an attacker deletes the challenged block and its Merkle sibling to reduce local storage.

The remaining two attackers target a hypothetical zero-knowledge-enabled DSN protocol, where only the data content (not the Merkle structure) is masked

[19] `https://anonymous.4open.science/r/eutxo-dsn/`.

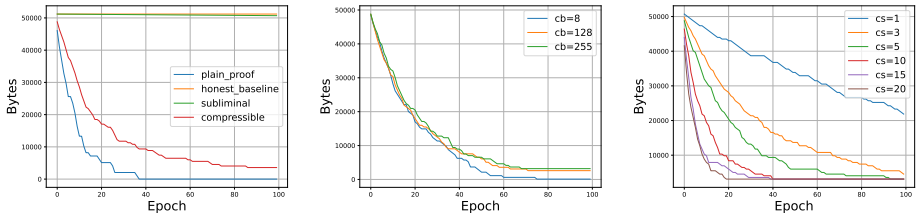
using a non-interactive Schnorr protocol. The subliminal channel attacker gradually leaks data by embedding small fragments (*e.g.*, a few bytes per challenge) into the proof randomness; and the compressible randomness attacker deliberately selects low-entropy nonces for Schnorr proofs, stores only these compressible values internally, and deletes the corresponding data blocks.

Experiments and results. We conduct three experiments, each runs for 100 epochs. The results are presented as follows.

- Overall attack comparison (Figure 12a): The *plain proof attacker* achieves the fastest storage reduction by offloading two data blocks (*i.e.*, the target and its sibling) per challenged index. The *compressible randomness attacker* also rapidly reduces its storage, discarding the whole block while retaining only a few bytes of low-entropy nonces per challenged index. However, this strategy may incur recomputation overhead if proof generation requires re-randomization. In contrast, the *subliminal channel attacker* leaks data at a much slower and linear rate due to a limited embedding capacity, *i.e.*, 8 bits per proof in our experiment. Increasing this rate is nontrivial, as it effectively requires solving a proof-of-work instance to find nonces that encode the desired data bits in the proof;
- Effect of nonce entropy for the compressible randomness attacker (Figure 12b): We vary the length of compressible randomness. A shorter nonce length enables fast storage reduction but introduces biased randomness, which makes the attack detectable. Notably, even with nearly full entropy, *i.e.*, using a 255-bit nonces, storage savings remain possible, as deleting data blocks still outweighs the cost of storing small nonces;
- Effect of challenge size for the compressible randomness attacker (Figure 12c): As noted in [19], larger challenge sets increase the chance of detecting deletion. However, without a proper proof system, they may also accelerate the rate at which storage can be safely discarded. In fact, let X denote the number of epochs until all n data blocks have been challenged at least once, where each epoch samples k indices uniformly at random, we can derive the cumulative distribution function of X as:

$$\Pr[X \leq x] = \frac{1}{n^{x \cdot k}} \sum_{i=0}^{n} (-1)^i \binom{n}{i} (n-i)^{x \cdot k}.$$

Final remark. Our toy experiments demonstrate that zero-knowledge is not a necessary requirement in DSN protocols when the primary goal is to prevent malicious provers from offloading internal storage with the aid of external systems, *e.g.*, a blockchain. Rather, it is the *non-malleability* and *succinctness* of proofs that underpin the security of existing DSN constructions [33,46], *i.e.*, not only due to their efficiency, but because they fundamentally restrict the ability to encode and leak data through proofs.



(a) Overall attack comparison. (b) Effect of nonce entropy. (c) Effect of challenge size.

Fig. 12: Internal storage over epochs under various attacks.

6 Conclusion

The final product of this work is a practical, adaptively secure and thoughtfully described DSN compatible with the EUTxO Model. Our protocol has the security fully specified and investigated in the rational setting. We consider our DSN a major step forward in the state-of-the-art for proof-of-space and variants thereof [16, 19, 20, 30, 33, 36, 46], given we propose novel security definitions such as *adaptive replication soundness*, *adaptive client-fairness*, and *server-fairness*, filling important security gaps in the literature. Furthermore, we back up our design choices by reporting practical attacks as experimental results. Finally, our proposed generic, yet practical, DSN protocol, in addition to its security framework can be seen as a blueprint for future concrete implementations.

References

1. Abdolmaleki, B., Campanelli, M., Dao, Q., Khoshakhlagh, H.: On the simulation-extractability of proof-carrying data. Cryptology ePrint Archive, Paper 2025/2037 (2025), `https://eprint.iacr.org/2025/2037`
2. Ateniese, G., Chen, L., Etemad, M., Tang, Q.: Proof of storage-time: Efficiently checking continuous data availability. In: NDSS 2020. The Internet Society (Feb 2020). https://doi.org/10.14722/ndss.2020.24427
3. Atzei, N., Bartoletti, M., Lande, S., Zunino, R.: A formal model of bitcoin transactions. In: Meiklejohn, S., Sako, K. (eds.) FC 2018. LNCS, vol. 10957, pp. 541–560. Springer, Berlin, Heidelberg (Feb / Mar 2018). https://doi.org/10.1007/978-3-662-58387-6_29
4. Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostáková, K., Maffei, M., Moreno-Sanchez, P., Riahi, S.: Generalized channels from limited blockchain scripts and adaptor signatures. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part II. LNCS, vol. 13091, pp. 635–664. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92075-3_22
5. Bellare, M., Neven, G.: Multi-signatures in the plain public-key model and a general forking lemma. In: Juels, A., Wright, R.N., De Capitani di Vimercati, S. (eds.) ACM CCS 2006. pp. 390–399. ACM Press (Oct / Nov 2006). https://doi.org/10.1145/1180405.1180453

6. Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. In: Garay, J.A., Gennaro, R. (eds.) CRYPTO 2014, Part II. LNCS, vol. 8617, pp. 276–294. Springer, Berlin, Heidelberg (Aug 2014). https://doi.org/10.1007/978-3-662-44381-1_16
7. Benet, J.: Ipfs - content addressed, versioned, p2p file system (2014), https://arxiv.org/abs/1407.3561
8. Boneh, D., Drake, J., Fisch, B., Gabizon, A.: Halo infinite: Proof-carrying data from additive polynomial commitments. In: Malkin, T., Peikert, C. (eds.) CRYPTO 2021, Part I. LNCS, vol. 12825, pp. 649–680. Springer, Cham, Virtual Event (Aug 2021). https://doi.org/10.1007/978-3-030-84242-0_23
9. Bowers, K.D., Juels, A., Oprea, A.: Proofs of retrievability: Theory and implementation. Cryptology ePrint Archive, Report 2008/175 (2008), https://eprint.iacr.org/2008/175
10. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: 42nd FOCS. pp. 136–145. IEEE Computer Society Press (Oct 2001). https://doi.org/10.1109/SFCS.2001.959888
11. Chakravarty, M.M.T., Chapman, J., MacKenzie, K., Melkonian, O., Jones, M.P., Wadler, P.: The extended UTXO model. In: Bernhard, M., Bracciali, A., Camp, L.J., Matsuo, S., Maurushat, A., Rønne, P.B., Sala, M. (eds.) FC 2020 Workshops. LNCS, vol. 12063, pp. 525–539. Springer, Cham (Feb 2020). https://doi.org/10.1007/978-3-030-54455-3_37
12. Crystal, M., Goren, G., Kominers, S.D.: Rationally analyzing shelby: Proving incentive compatibility in a decentralized storage network (2025), https://arxiv.org/abs/2510.11866
13. Dai, W., Okamoto, T., Yamamoto, G.: Stronger security and generic constructions for adaptor signatures. In: Isobe, T., Sarkar, S. (eds.) INDOCRYPT 2022. LNCS, vol. 13774, pp. 52–77. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22912-1_3
14. Damgård, I., Ganesh, C., Orlandi, C.: Proofs of replicated storage without timing assumptions. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019, Part I. LNCS, vol. 11692, pp. 355–380. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26948-7_13
15. Dziembowski, S., Eckey, L., Faust, S.: FairSwap: How to fairly exchange digital goods. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 967–984. ACM Press (Oct 2018). https://doi.org/10.1145/3243734.3243857
16. Dziembowski, S., Faust, S., Kolmogorov, V., Pietrzak, K.: Proofs of space. In: Gennaro, R., Robshaw, M.J.B. (eds.) CRYPTO 2015, Part II. LNCS, vol. 9216, pp. 585–605. Springer, Berlin, Heidelberg (Aug 2015). https://doi.org/10.1007/978-3-662-48000-7_29
17. Faonio, A., Fiore, D., Russo, L.: Real-world universal zkSNARKs are non-malleable. In: ACM CCS 2024. pp. 3138–3151. ACM Press (Nov 2024). https://doi.org/10.1145/3658644.3690351
18. Feige, U., Shamir, A.: Zero knowledge proofs of knowledge in two rounds. In: Brassard, G. (ed.) CRYPTO'89. LNCS, vol. 435, pp. 526–544. Springer, New York (Aug 1990). https://doi.org/10.1007/0-387-34805-0_46
19. Fisch, B.: PoReps: Proofs of space on useful data. Cryptology ePrint Archive, Report 2018/678 (2018), https://eprint.iacr.org/2018/678
20. Fisch, B.: Tight proofs of space and replication. In: Ishai, Y., Rijmen, V. (eds.) EUROCRYPT 2019, Part II. LNCS, vol. 11477, pp. 324–348. Springer, Cham (May 2019). https://doi.org/10.1007/978-3-030-17656-3_12

21. Fisch, B., Silas, S.: Weak compression and (in)security of rational proofs of storage. Cryptology ePrint Archive, Report 2018/514 (2018), https://eprint.iacr.org/2018/514
22. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953 (2019), https://eprint.iacr.org/2019/953
23. Garay, J.A., Kiayias, A., Leonardos, N.: The bitcoin backbone protocol: Analysis and applications. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015, Part II. LNCS, vol. 9057, pp. 281–310. Springer, Berlin, Heidelberg (Apr 2015). https://doi.org/10.1007/978-3-662-46803-6_10
24. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. In: Stern, J. (ed.) EUROCRYPT'99. LNCS, vol. 1592, pp. 295–310. Springer, Berlin, Heidelberg (May 1999). https://doi.org/10.1007/3-540-48910-X_21
25. Gerhart, P., Schröder, D., Soni, P., Thyagarajan, S.A.K.: Foundations of adaptor signatures. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part II. LNCS, vol. 14652, pp. 161–189. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58723-8_6
26. Gilad, Y., Hemo, R., Micali, S., Vlachos, G., Zeldovich, N.: Algorand: Scaling byzantine agreements for cryptocurrencies. Cryptology ePrint Archive, Report 2017/454 (2017), https://eprint.iacr.org/2017/454
27. Goldwasser, S., Micali, S., Rivest, R.L.: A digital signature scheme secure against adaptive chosen-message attacks. SIAM Journal on Computing **17**(2), 281–308 (Apr 1988)
28. Goren, G., Hariri, A., Hartley, T.D.R., Kappiyoor, R., Spiegelman, A., Zmick, D.: Shelby: Decentralized storage designed to serve (2025), https://arxiv.org/abs/2506.19233
29. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part II. LNCS, vol. 9666, pp. 305–326. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49896-5_11
30. Juels, A., Kaliski, Jr., B.S.: Pors: proofs of retrievability for large files. In: Ning, P., De Capitani di Vimercati, S., Syverson, P.F. (eds.) ACM CCS 2007. pp. 584–597. ACM Press (Oct 2007). https://doi.org/10.1145/1315245.1315317
31. Katz, J., Lindell, Y.: Introduction to Modern Cryptography, Second Edition. CRC Press (2014), https://www.crcpress.com/Introduction-to-Modern-Cryptography-Second-Edition/Katz-Lindell/p/book/9781466570269
32. Kilian, J.: A note on efficient zero-knowledge proofs and arguments (extended abstract). In: 24th ACM STOC. pp. 723–732. ACM Press (May 1992). https://doi.org/10.1145/129712.129782
33. Labs, P.: Filecoin: A decentralized storage network (2017), https://filecoin.io/filecoin.pdf
34. Libert, B.: Simulation-extractable KZG polynomial commitments and applications to HyperPlonk. In: Tang, Q., Teague, V. (eds.) PKC 2024, Part II. LNCS, vol. 14602, pp. 68–98. Springer, Cham (Apr 2024). https://doi.org/10.1007/978-3-031-57722-2_3
35. Micali, S.: CS proofs (extended abstracts). In: 35th FOCS. pp. 436–453. IEEE Computer Society Press (Nov 1994). https://doi.org/10.1109/SFCS.1994.365746

36. Moran, T., Orlov, I.: Simple proofs of space-time and rational proofs of storage. In: Boldyreva, A., Micciancio, D. (eds.) CRYPTO 2019, Part I. LNCS, vol. 11692, pp. 381–409. Springer, Cham (Aug 2019). https://doi.org/10.1007/978-3-030-26948-7_14
37. Pass, R., Rosen, A.: New and improved constructions of non-malleable cryptographic protocols. In: Gabow, H.N., Fagin, R. (eds.) 37th ACM STOC. pp. 533–542. ACM Press (May 2005). https://doi.org/10.1145/1060590.1060670
38. Poelstra, A.: Scriptless scripts (2017), https://lists.launchpad.net/mimblewimble/msg00086.html
39. Querejeta-Azurmendi, I.: Plutus support for pairings over bls12-381 (2022), https://cips.cardano.org/cip/CIP-0381
40. Storj Labs, I.: Storj: A decentralized cloud storage network framework (2018), https://static.storj.io/storjv3.pdf
41. Tas, E.N., Seres, I.A., Zhang, Y., Melczer, M., Kelkar, M., Bonneau, J., Nikolaenko, V.: Atomic and fair data exchange via blockchain. In: ACM CCS 2024. pp. 3227–3241. ACM Press (Nov 2024). https://doi.org/10.1145/3658644.3690248
42. team, S.: Swarm: Storage and communication infrastructure for a self-sovereign digital society (2021), https://www.ethswarm.org/swarm-whitepaper.pdf
43. Vellekoop, T.: plutus-plonk-example (2024), https://github.com/perturbing/plutus-plonk-example
44. Vorick, D., Champine, L.: Sia: Simple decentralized storage (2014)
45. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger (2014)
46. Xu, M., Zhang, J., Guo, H., Cheng, X., Yu, D., Hu, Q., Li, Y., Wu, Y.: FileDES: A secure, scalable and succinct decentralized encrypted storage network. Cryptology ePrint Archive, Report 2024/182 (2024), https://eprint.iacr.org/2024/182

A Auxiliary Definitions

This section provides formal definitions in addition to Section 2.

A.1 Fundamental Cryptographic Primitives

Here recalls the syntax and security of a public-key encryption scheme, a signature scheme, a hard relation, and a non-interactive argument protocol[20].

Symmetric encryption. A symmetric encryption scheme SKE consists of a tuple of algorithms $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ defined as follows.

- $\mathsf{KGen}(1^\lambda)$ takes as input the security parameter λ. It outputs a private key K;
- $\mathsf{Enc}(K, m)$ takes as input a key K and a message $m \in \{0,1\}^*$. It outputs a ciphertext ct;
- $\mathsf{Dec}(K, ct)$ is deterministic, and takes as input a key K and a ciphertext ct. It outputs a message $m \in \{0,1\}^*$ or $\perp$.

The correctness and the indistinguishability under chosen plaintext attacks (IND-CPA) [31] are given as follows.

Definition 10 (Correctness). *A symmetric encryption scheme satisfies correctness if, the following holds for any $\lambda > 0$, $m \in \{0,1\}^*$ and $K \leftarrow \mathsf{KGen}(1^\lambda)$:*

$$\Pr\left[\mathsf{Dec}(K, \mathsf{Enc}(K, m)) = m\right] = 1.$$

Definition 11 (IND-CPA). *A symmetric encryption scheme is IND-CPA secure if, for any PPT adversary $\mathcal{A}$ with access to an encryption oracle $\mathcal{O}_{\mathsf{Enc}}(K, \cdot)$, the following probability*

$$\left| \Pr\left[b' = b \;\middle|\; \begin{array}{c} (m_0, m_1) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Enc}}(K,\cdot)}(1^\lambda) \ s.t.\ |m_0| = |m_1|; \\ b \xleftarrow{\$} \{0,1\}; ct_b \leftarrow \mathsf{Enc}(K, m_b); \\ b' \leftarrow \mathcal{A}^{\mathsf{Enc}_K(\cdot)}(ct_b) \end{array} \right] - \frac{1}{2} \right|$$

is $\mathsf{negl}(\lambda)$, for any $\lambda > 0$ and $K \leftarrow \mathsf{KGen}(1^\lambda)$.

Signature scheme. A signature scheme SIG consists of a tuple of algorithms $(\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ defined as follows:

- $\mathsf{KGen}(1^\lambda)$ takes as input the security parameter λ. It outputs a secret-public key pair (sk, pk);
- $\mathsf{Sign}(sk, m)$ takes as input a secret key sk and a message m. It outputs a signature σ on m under sk; and
- $\mathsf{Vrfy}(pk, m, \sigma)$ is deterministic, and takes as input a public key pk, a message m, and a signature σ. It outputs 1 if σ is valid, or 0 otherwise.

[20] The algorithm notations are confined to their respective sub-subsections for clarity.

The correctness and the (strong) existential unforgeability under adaptive chosen message attacks (EUF/SUF-CMA) [27] are given as follows.

Definition 12 (Correctness). *A signature scheme satisfies correctness if the following*

$$\Pr\left[\mathsf{Vrfy}(pk, m, \mathsf{Sign}(sk, m)) = 1\right] = 1$$

holds for any $\lambda > 0$, message m, and $(sk, pk) \leftarrow \mathsf{KGen}(1^\lambda)$.

Definition 13 (EUF-CMA (*resp.*, SUF-CMA)). *A signature scheme is EUF-CMA (resp., SUF-CMA) secure if, for any PPT adversary $\mathcal{A}$ with access to a signing oracle $\mathcal{O}_{\mathsf{Sign}}(sk, \cdot)$, which on receiving m from $\mathcal{A}$, returns $\sigma \leftarrow \mathsf{Sign}(sk, m)$ to $\mathcal{A}$ and records m (resp., (m, σ)) in a queried set Q, the probability*

$$\Pr\left[\mathsf{Vrfy}(m^*, \sigma^*, pk) = 1 \wedge m^* \notin Q(resp., (m^*, \sigma^*) \notin Q))|(m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Sign}}}(pk)\right]$$

is $\mathsf{negl}(\lambda)$, for any $\lambda > 0$ and $(sk, pk) \leftarrow \mathsf{KGen}(1^\lambda)$.

Hard Relation Primitive. We adopt the description of a relation primitive from [13]. A relation primitive $\mathsf{R} := (\mathsf{Gen}, \mathsf{Vrfy})$ is associated with an NP relation $\mathcal{R} \subseteq \{0,1\}^* \times \{0,1\}^*$, where

- $\mathsf{Gen}(1^\lambda)$ is a PPT algorithm that takes as input the security parameter λ, and outputs a pair $(x, w) \in \mathcal{R}$; and
- $\mathsf{Vrfy}(x, w)$ is a deterministic polynomial time algorithm that outputs 1 if, and only if, the instance-witness pair $(x, w) \in \mathcal{R}$.

The language $L_{\mathcal{R}}$ for the relation $\mathcal{R}$ is defined as $\{x \in \{0,1\}^* | \exists w \in \{0,1\}^* \ s.t. \ (x, w) \in \mathcal{R}\}$. We recall the hardness and unique witness for the relation $\mathcal{R}$.

Definition 14 (Hardness). *A relation is hard if, for any PPT adversary $\mathcal{A}$ and positive polynomial $q(\lambda) \in \mathbb{N}$, the following probability*

$$\Pr\left[(x_{i^*}, w^*) \in \mathcal{R} \;\middle|\; \begin{array}{c} For\ i \in [q]\ do\ (x_i, \cdot) \leftarrow \mathsf{Gen}(1^\lambda); \\ (i^*, w^*) \leftarrow \mathcal{A}(\{x_i\}_{i \in [q]}) \end{array}\right]$$

is $\mathsf{negl}(\lambda)$ for any $\lambda > 0$.

Definition 15 (Unique Witness). *A relation has unique witness if, for any PPT adversary $\mathcal{A}$, the following probability*

$$\Pr\left[(x, w) \in \mathcal{R} \wedge (x, w') \in \mathcal{R} \wedge w \neq w' | (x, w, w') \leftarrow \mathcal{A}(1^\lambda)\right]$$

is $\mathsf{negl}(\lambda)$ for any $\lambda > 0$.

Non-interactive arguments of knowledge. Let $\mathcal{R}$ be an NP relation, parameterized by λ. A non-interactive argument protocol $\Pi_{\mathcal{R}}$ for $\mathcal{R}$ consists of a tuple of algorithms $(\mathsf{Gen}, \mathsf{Prove}, \mathsf{Vrfy})$ defined as follows

- $\mathsf{Gen}(1^\lambda, \mathcal{R})$ takes as input the security parameter λ. It outputs a common reference string crs and a trapdoor td;
- $\mathsf{Prove}(\mathsf{crs}, x, w)$ takes as input crs, an instance x, and a witness w. It outputs an argument π; and
- $\mathsf{Vrfy}(\mathsf{crs}, x, \pi)$ is deterministic, and takes as input crs, an instance x, and an argument π. It outputs 1 if π is valid, or 0 otherwise.

The basic security properties are completeness and knowledge soundness [18][21]. We further introduce zero-knowledge [18] and succinctness [32, 35] as modular properties. That is, incorporating zero-knowledge yields a non-interactive zero-knowledge proof of knowledge (NIZKPoK) protocol (with computational knowledge soundness). Whereas, combining both results in a succinct non-interactive argument of knowledge (zkSNARK). Additionally, we present the simulation-extractability, an extended form of knowledge soundness that implies non-malleability [37], a crucial property for achieving adaptive security.

Definition 16 (Completeness). *A non-interactive argument protocol for a relation $\mathcal{R}$ satisfies completeness, if*

$$\Pr[\mathsf{Vrfy}(\mathsf{crs}, x, \pi) = 1 | \pi \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)] = 1$$

holds for any $\lambda > 0$, any $x \in L_{\mathcal{R}}$, w s.t. $(x, w) \in \mathcal{R}$, and any $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Gen}(1^\lambda, \mathcal{R})$.

Definition 17 (Computational Knowledge Soundness). *A non-interactive argument protocol for $\mathcal{R}$ is an argument of knowledge if, for any* PPT *adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}^{\mathcal{A}}$ (with rewinding black-box access to $\mathcal{A}$) s.t.*

$$\Pr\left[\mathsf{Vrfy}(\mathsf{crs}, x, \pi) = 1 \wedge (x, w) \notin \mathcal{R} \;\middle|\; \begin{array}{c}(x, \pi) \leftarrow \mathcal{A}(\mathsf{crs}); \\ w \leftarrow \mathcal{E}^{\mathcal{A}}(\mathsf{crs})\end{array}\right]$$

is $\mathsf{negl}(\lambda)$ for any $\lambda > 0$ and $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Gen}(1^\lambda, \mathcal{R})$.

Definition 18 (Statistical Zero-Knowledge). *A non-interactive argument protocol for $\mathcal{R}$ is statistical zero-knowledge if, for any PPT adversary $\mathcal{A}$, there exists a PPT simulator* Sim *s.t. the following distributions*

$$\mathcal{D}_0 \coloneqq \{\pi_0 \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)\} \text{ and } \mathcal{D}_1 \coloneqq \{\pi_1 \leftarrow \mathsf{Sim}(\mathsf{crs}, \mathsf{td}, x)\}$$

are statistically close for any $\lambda > 0$, $(x, w) \in \mathcal{R}$, and $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Gen}(1^\lambda, \mathcal{R})$.

[21] We adopt knowledge soundness rather than the weaker notion of soundness, hence characterizing the protocol as a non-interactive argument of knowledge.

Definition 19 (Preprocessing Succinctness). *A non-interactive argument protocol satisfies preprocessing succinctness (i.e., without constraints on* Gen*) if the verifier runs in* $\mathsf{poly}(\lambda, |x|)$ *and the proof size* $|\pi| \leq \mathsf{poly}(\lambda)$ *for any* $\lambda > 0$*,* $(x, w) \in \mathcal{R}$*,* $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Gen}(1^\lambda, \mathcal{R})$*, and* $\pi \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)$*.*

Definition 20 (Simulation-Extractability). *A non-interactive argument protocol for* $\mathcal{R}$ *satisfies simulation-extractable knowledge soundness if, for any* PPT *adversary* $\mathcal{A}$*, there exists a PPT* $(\mathsf{Sim}, \mathcal{E}^{\mathcal{A}})$ *s.t. the following properties hold for any* $\lambda > 0$ *and* $(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Gen}(1^\lambda, \mathcal{R})$*.*

- *The distribution from the execution of* Prove *and that from* Sim*, as given in the zero-knowledge definition, are statistically close (with* $\mathcal{A}$ *as the distinguisher);*
- *For any PPT adversary with access to a simulation-proof oracle* $\mathcal{O}_{\mathsf{Sim}}(\mathsf{crs}, \mathsf{td}, \cdot)$*, which on receiving* x *from* $\mathcal{A}$*, returns* $\pi \leftarrow \mathsf{Sim}(\mathsf{crs}, \mathsf{td}, x)$ *to* $\mathcal{A}$ *and records* (x, π) *in a queried set* Q*, the following probability, is* $\mathsf{negl}(\lambda)$*,*

$$\Pr\left[\begin{array}{c} \mathsf{Vrfy}(\mathsf{crs}, x, \pi) = 1 \wedge \\ (x, w) \notin \mathcal{R} \wedge (x, \pi) \notin Q \end{array} \middle| \begin{array}{c} (x, \pi) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Sim}}}(\mathsf{crs}); \\ w \leftarrow \mathcal{E}^{\mathcal{A}}(\mathsf{crs}, Q) \end{array}\right].$$

A.2 Adaptor Signatures

Let SIG and R be a signature scheme and a relation primitive for an NP relation $\mathcal{R}$, as defined above. We present the syntax and security of an adaptor signature scheme for SIG and R.

Syntax. An adaptor signature scheme AS for an NP relation $\mathcal{R}$ (with instance-witness pairs generated by $\mathsf{R.Gen}$) extends $\mathsf{SIG} = (\mathsf{KGen}, \mathsf{Sign}, \mathsf{Vrfy})$ with a tuple of algorithms $(\mathsf{pSign}, \mathsf{pVrfy}, \mathsf{Adapt}, \mathsf{wExt})$, in which pSign is probabilistic and the others are deterministic. Given $(sk, pk) \leftarrow \mathsf{KGen}(1^\lambda)$ and $(x, w) \leftarrow \mathsf{R.Gen}(1^\lambda)$, the extension is as follows.

- $\mathsf{pSign}(sk, m, x)$ takes as input a secret key sk, a message m, and an instance x. It outputs a pre-signature $\hat{\sigma}$;
- $\mathsf{pVrfy}(pk, m, \hat{\sigma}, x)$ takes as input a public key pk, a pre-signature $\hat{\sigma}$, and an instance x. It outputs 1 if $\hat{\sigma}$ is a valid, or 0 otherwise;
- $\mathsf{Adapt}(pk, \hat{\sigma}, w)$ outputs a signature σ adapted from a public key pk, a pre-signature $\hat{\sigma}$ and its corresponding witness w; and
- $\mathsf{wExt}(\sigma, \hat{\sigma}, x)$ outputs the witness w extracted from a signature σ, its pre-signature $\hat{\sigma}$, and the instance x.

Security. We recall the experiments and definitions of correctness, pre-signature adaptability, extractability, and unique extractability from [13][22].

[22] Note that the standard extractability notion for adaptor signatures is the (strong) full extractability. Assuming the hardness of $\mathcal{R}$, the stronger variant is implied by the combination of extractability and unique extractability (cf. [13, Theorem 3.4] and stated below in Lemma 1).

$\mathsf{Expt}^{\text{correct}}_{\mathsf{AS},m,\mathsf{KGen},\mathsf{R.Gen}}(\lambda)$:

$\hat{\sigma} \leftarrow \mathsf{pSign}(sk, m, x)$;

Assert $b_1 \leftarrow \mathsf{pVrfy}(pk, m, \hat{\sigma}, x)$;

$\sigma \leftarrow \mathsf{Adapt}(pk, \hat{\sigma}, w)$;

$b_2 \leftarrow \mathsf{Vrfy}(pk, m, \sigma)$;

$w \leftarrow \mathsf{wExt}(\hat{\sigma}, \sigma, x)$;

$b_3 \leftarrow (x, w) \in \mathcal{R}$;

Return $b_1 \wedge b_2 \wedge b_3$

$\mathsf{Expt}^{\text{adapt}}_{\mathsf{AS},\mathcal{A},\mathsf{KGen}}(\lambda)$:

$(m, \hat{\sigma}, (x, w)) \leftarrow \mathcal{A}(pk)$;

Assert $(x, w) \in \mathcal{R} \wedge \mathsf{pVrfy}(pk, m, \hat{\sigma}, x)$;

Return $\mathsf{Vrfy}(pk, m, \mathsf{Adapt}(pk, \hat{\sigma}, w))$

$\mathsf{Expt}^{\text{ext}}_{\mathsf{AS},\mathcal{A},\mathsf{KGen}}(\lambda)$:

$(m^*, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{pSign}}}(pk)$;

Assert $\mathsf{Vrfy}(pk, m^*, \sigma)$;

Return $\forall(\hat{\sigma}, x) \in Q[m^*]$:

$(x, \mathsf{wExt}(\hat{\sigma}, \sigma, x)) \notin \mathcal{R}$

$\mathsf{Expt}^{\text{uni-ext}}_{\mathsf{AS},\mathcal{A},\mathsf{KGen}}(\lambda)$:

$(m, \hat{\sigma}, \sigma, \sigma', x) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{pSign}}}(pk)$;

Assert $\mathsf{pVrfy}(pk, m, \hat{\sigma}, x) \wedge \mathsf{Vrfy}(pk, m, \sigma) \wedge \mathsf{Vrfy}(pk, m, \sigma') \wedge \sigma \neq \sigma'$;

$w \leftarrow \mathsf{wExt}(\hat{\sigma}, \sigma, x)$; $w' \leftarrow \mathsf{wExt}(\hat{\sigma}, \sigma', x)$;

Return $(x, w) \in \mathcal{R} \wedge (x, w') \in \mathcal{R}$

Fig. 13: Experiments for adaptor signature scheme AS.

Definition 21 (Correctness). *An adaptor signature scheme is correct, if*

$$\Pr\left[\mathsf{Expt}^{\text{correct}}_{\mathsf{AS},m,\mathsf{KGen},\mathsf{R.Gen}}(\lambda) = 1\right] = 1$$

for any $\lambda{>}0$*,* m*,* $(sk, pk){\leftarrow}\mathsf{KGen}(1^\lambda)$*, and* $(x, w){\leftarrow}\mathsf{R.Gen}(1^\lambda)$*.*

Definition 22 (Pre-signature Adaptability). *An adaptor signature scheme satisfies perfect pre-signature adaptability if for any PPT adversary* $\mathcal{A}$*:*

$$\Pr\left[\mathsf{Expt}^{\text{adapt}}_{\mathsf{AS},\mathcal{A},\mathsf{KGen}}(\lambda) = 1\right] = 1$$

for any $\lambda > 0$ *and* $(sk, pk) \leftarrow \mathsf{KGen}(1^\lambda)$*.*

Let $\mathcal{O}_{\mathsf{pSign}}(sk, \cdot, \cdot)$ be a pre-signing oracle. On receiving a query (m, x) from an adversary $\mathcal{A}$, it returns $\hat{\sigma} \leftarrow \mathsf{pSign}(sk, m, x)$ to $\mathcal{A}$ and records $(\hat{\sigma}, x)$ in a queried set Q *w.r.t.* m, denoted by a mapping form $(\hat{\sigma}, x) \in Q[m]$.

Definition 23 (Extractability and Unique Extractability). *An adaptor signature scheme satisfies extractability (resp., unique extractability) if, for any PPT adversary* $\mathcal{A}$ *with access to* $\mathcal{O}_{\mathsf{pSign}}(sk, \cdot, \cdot)$*, the following probability*

$$\Pr\left[\mathsf{Expt}^{\text{ext }(resp.,\text{uni-ext})}_{\mathsf{AS},\mathcal{A},\mathsf{KGen}}(\lambda) = 1\right]$$

is $\mathsf{negl}(\lambda)$*, for any* $\lambda > 0$ *and* $(sk, pk) \leftarrow \mathsf{KGen}(1^\lambda)$*.*

Lemma 1 (Implication of strong full extractability). *Let* AS *be a canonical adaptor signature scheme for a* hard *relation* $\mathcal{R}$*. If* AS *satisfies* extractability *and* unique extractability*, then it satisfies strong full extractability.*

B Supplementary Materials Related to PoRep

This section provides supplementary materials for PoRep [19].

B.1 Auxiliary PoRep Definitions

We begin by recalling the original replication soundness definition [19], which is given by the experiment in Figure 14.

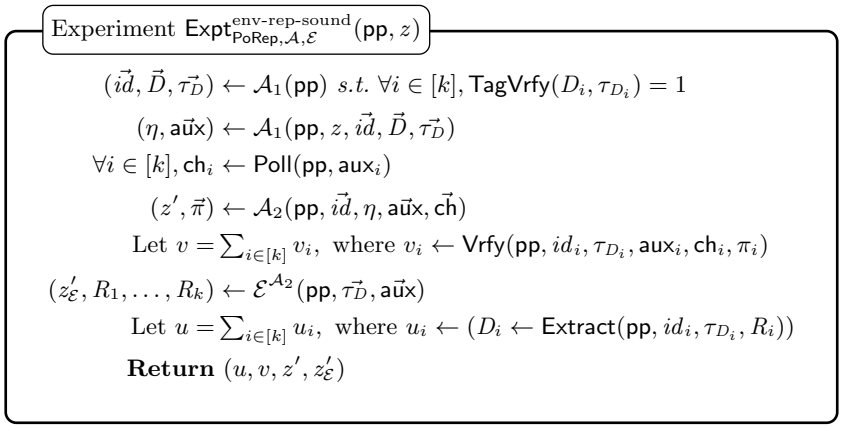


Fig. 14: The experiment for replication soundness with an auxiliary environment string z. The adversary $\mathcal{A}$ receives (pp, z). The experiment ends by returning the number of successfully answered challenges from $\mathcal{A}$ and that of successfully extracted retrievable replica from extractor $\mathcal{E}^{\mathcal{A}_2}$, respectively. In addition, it returns the encoded environment strings produced by $\mathcal{A}$ and $\mathcal{E}^{\mathcal{A}_2}$, *i.e.*, $(z', z'_{\mathcal{E}})$.

Definition 24 (Replication Soundness with z). *A PoRep is ϵ-replication sound if, for any adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, where $\mathcal{A}_1$ runs in $O(\mathsf{poly}(\lambda))$ and $\mathcal{A}_2$ runs in parallel time at most t, there exists an extractor $\mathcal{E}^{\mathcal{A}_2}$ s.t. the following*

$$\Pr\left[\begin{array}{c} Z' \approx_c Z'_{\mathcal{E}} \wedge \\ u < v \vee (|\eta| < (1-\epsilon)v|R|) \end{array} \middle| (u, v) \leftarrow \mathsf{Expt}^{\text{env-rep-sound}}_{\mathsf{PoRep},\mathcal{A},\mathcal{E}}(\mathsf{pp}) \right]$$

is $\mathsf{negl}(\lambda)$ for any $\lambda, t > 0$ and $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, t)$. Here, $Z' \approx_c Z'_{\mathcal{E}}$ denotes that the distributions of the random variables corresponding to z' (i.e., Z') and $z'_{\mathcal{E}}$ (i.e., $Z'_{\mathcal{E}}$) are computationally indistinguishable.

A key property underlying replication soundness, both in their work and in ours, is the incompressibility of data (or its replicas). Accordingly, we recall the definition of time bounded incompressibility below.

Definition 25 (Time Bounded Incompressibility). *A data source $\mathcal{S}$ is (t_1, t_2)-incompressible to k bits if there does not exist (for any constant ξ) a ξ-lossy strong compression scheme* $(\mathsf{Encode}, \mathsf{Decode})$ *where* Encode *runs in parallel time t_1 and* Decode *runs in parallel time t_2 that compresses $\mathcal{S}$ to $k - \log(1/\xi)$ bits.*

The original definition (Definition 24) models potential composition via the additional string z provided to the adversary by an external environment. The rationale is as follows: if any adversary can compress z using an incompressible string η (*i.e.*, the replicas), then it must be exploiting only the compressibility of z. That is, access to z with η should not provide any advantage. This intuition is formalized as the KoSC assumption proposed in [21], which is also presented below.

Assumption 1 (Knowledge of Strong Compression (KoSC))**.** *Let $\mathcal{S}$ and $\mathcal{X}$ be any two samplable sources over $\{0,1\}^N$ s.t. $\mathcal{S}$ is (t_1, t_2)-incompressible to $(1-\epsilon)N$ bits for some $\epsilon < 1$. For any compression scheme* $(\mathsf{Encode}, \mathsf{Decode})$ *that compresses $\mathcal{S} \times \mathcal{X}$ to M bits where* Encode *runs in parallel time t_1 or* Decode *runs in parallel time t_2, there exists a strong compression scheme* $(\mathsf{Encode}^*, \mathsf{Decode}^*)$ *running in parallel time t_1 and t_2, respectively, that compresses $\mathcal{X}$ to length $M - (1-\epsilon)N$ bits sampling seeds from $\mathcal{S}$. Moreover, for any algorithm $\mathcal{A}$ that on inputs $(\mathcal{S}, N)$ outputs* $(\mathsf{Encode}, \mathsf{Decode})$*, there exists an efficient extractor which observes $\mathcal{A}$'s internal state and outputs* $(\mathsf{Encode}^*, \mathsf{Decode}^*)$.

Finally, we note that the environment string is omitted in our setting (Definition 3 and 7), as it does not adequately capture the composition requirement. Although its inclusion introduces an additional assumption, *i.e.*, knowledge of strong compression, it offers limited benefit towards achieving formal composability [10], which remains a highly non-trivial task due to the following challenges: (1) the current replication soundness requires its extractor to have rewinding black-box access to the adversary; and more fundamentally, (2) the UC framework abstracts away resource constraints; as a result, it lacks a notion of space, and the plain UC framework does not model physical time, which is crucial for PoRep protocols with timing assumptions.

B.2 Implicit PoRC and PoSpace Schemes

The implicit PoRC and PoSpace derived from a PoRep scheme $\mathsf{PoRep} = (\mathsf{Setup}, \mathsf{Preproc}, \mathsf{TagVrfy}, \mathsf{Replicate}, \mathsf{Extract}, \mathsf{Poll}, \mathsf{Prove}, \mathsf{Vrfy})$ are presented here. The presentation begins with the implicit PoRC.

- $\mathsf{PoRC.Commit}(\mathsf{pp}, D, \tau_D)$: Run $(R, \mathsf{aux}) \leftarrow \mathsf{PoRep.Replicate}(id, D, \tau_D)$, set the opening as $o = (id, R)$, and output the commitment $c = (\tau_D, \mathsf{aux})$;
- $\mathsf{PoRC.Open}(\mathsf{pp}, c, o)$: Parse $o = (id, R)$ and $c = (\tau_D, \mathsf{aux})$. $D \leftarrow \mathsf{PoRep.Extract}$ (id, D, aux, R). If $\mathsf{PoRep.TagVrfy}(D, \tau_D) = 0$, output $\perp$, or output (D, R);
- $\mathsf{PoRC.Prove}(\mathsf{pp}, c, o, \mathsf{ch})$: Parse $o = (id, R)$ and $c = (\tau_D, \mathsf{aux})$. Generate the PoRep proof $\pi' \leftarrow \mathsf{PoRep.Prove}(id, R, \mathsf{aux}, \mathsf{ch})$ and output $\pi \coloneqq (\pi', id)$;
- $\mathsf{PoRC.Vrfy}(\mathsf{pp}, c, \mathsf{ch}, \pi)$: Parse $c = (\tau_D, \mathsf{aux})$ and $\pi = (\pi', id)$. Output $\mathsf{PoRep.}$ $\mathsf{Vrfy}(id, \tau_D, \mathsf{aux}, \mathsf{ch}, \pi')$.

Definition 26 (PoRC Binding). *A PoRC scheme is binding if, for any PPT adversary $\mathcal{A}$, the probability*

$$\Pr\left[m \neq m' \neq \perp \;\middle|\; \begin{array}{c}(c, o, o') \leftarrow \mathcal{A}(\mathsf{pp});\\ m \leftarrow \mathsf{PoRC.Open}(\mathsf{pp}, c, o);\\ m' \leftarrow \mathsf{PoRC.Open}(\mathsf{pp}, c, o')\end{array}\right]$$

is $\mathsf{negl}(\lambda)$*, for any* $\lambda > 0$ *and* $\mathsf{pp} \leftarrow \mathsf{PoRC.Setup}(1^\lambda)$*.*

Definition 27 (PoRC Soundness). *A PoRC scheme is sound if it is binding and, additionally, for any $\lambda > 0$, there exists a PPT extractor $\mathcal{E}$ (with rewinding*[23] *black-box access to $\mathcal{A}$), s.t. for any PPT adversary $\mathcal{A}$, the probability*

$$\Pr\left[\begin{array}{c}\mathsf{PoRC.Vrfy}(\mathsf{pp}, c, \mathsf{ch}, \pi) = 1 \wedge\\ (m' \neq m \vee m_{\mathcal{E}} \neq m)\end{array} \;\middle|\; \begin{array}{c}(c, o, \mathsf{st}) \leftarrow \mathcal{A}_1(\mathsf{pp}, m);\\ m' \leftarrow \mathsf{PoRC.Open}(\mathsf{pp}, c, o);\\ \mathsf{ch} \xleftarrow{\$} \mathcal{CH}; \pi \leftarrow \mathcal{A}_2(\mathsf{pp}, \mathsf{st}, c, \mathsf{ch});\\ m_{\mathcal{E}} \leftarrow \mathcal{E}^{\mathcal{A}_2}(\mathsf{pp})\end{array}\right]$$

is $\mathsf{negl}(\lambda)$*, for any* $\mathsf{pp} \leftarrow \mathsf{PoRC.Setup}(1^\lambda)$ *and message m from the message space. Moreover, the challenge* ch *is sampled uniformly at random from the challenge space $\mathcal{CH}$.*

The following outlines the full construction of the induced PoSpace, and provides the lemma that establishes the relation between PoSpace and PoRep [19, Lemma 1].

- $\mathsf{PoSpace.Setup}$: Run $\mathsf{PoRep.Setup}(\lambda, t)$;
- $\mathsf{PoSpace.Initialization}$: The prover runs $(R, \mathsf{aux}) \leftarrow \mathsf{PoRep.Replicate}(id, D, \tau_D)$, and outputs $\tau_D = \mathsf{aux}$ and $S = R$;
- $\mathsf{PoSpace.Execution}$: The verifier's challenge is $\mathsf{ch} \leftarrow \mathsf{PoRep.Poll}(\mathsf{aux})$, the prover responds with $\pi \leftarrow \mathsf{PoRep.Prove}(id, R, \mathsf{aux}, \mathsf{ch})$, and the verifier runs $\mathsf{PoRep.Vrfy}(id, \tau_D, \mathsf{aux}, \mathsf{ch}, \pi)$.

Lemma 2 (PoSpace is Necessary). *If a PoRep is (μ, ϵ)-replication sound, then it is a $((1-\epsilon)kN, t, \mu)$-sound PoSpace, where $N = |D|$, i.e., there exists* no *PoSpace adversary the passes the verification with probability greater than μ using only $(1-\epsilon)kN$ storage.*

Finally, we recall the sufficient condition for a (μ, ϵ)-replication sound PoRep, which gives a generic construction under the KoSC assumption [19, Lemma 2].

Lemma 3 (Sufficient Conditions for Replication Soundness). *Given the KoSC assumption (Assumption 1), a PoRep construction with setup parameters (λ, t) satisfies (μ, ϵ)-replication soundness if the following conditions hold for some integer c:*

[23] As noted in [19], the extractor is allowed to reissue challenges and rewind $\mathcal{A}$'s internal randomness.

– *It is correct (Definition 2);*
– *It is a μ-sound PoRC (Definition 27), and the extractor $\mathcal{E}$ in the definition has parallel runtime that exceeds that of the adversary $\mathcal{A}_2$ by a factor c;*
– *k independent instantiations of the PoRep form a parallel $((1-\epsilon)kN, (c+1)t, \mu)$-sound PoSpace.*

C Auxiliary Protocol Details and Illustrations

This section presents syntax-level design configurations, discusses a weaker variant of client-fairness and privacy concerns, and provides illustrative diagrams that complement the construction presented in Section 4.

C.1 Configurations

As introduced in Section 3.2, our protocol is highly configurable. Here, we examine three key aspects: (1) payment and collateral settings *w.r.t.* incentive models; (2) scalability optimizations for the preservation phase; and (3) retrieval policies and configuration options.

Incentive model. We begin with an observation on fairness in the data retrieval phase. The guarantees inherently permit a lose-lose outcome, where the client loses her data and server forfeits her collateral.

More critically, the current notion of server-fairness may be insufficient in practical deployments, due to a key difference between our *retrieval* model and the original data *exchange* protocol [41]. Specifically, a client who delegates identical data to multiple servers may retrieve the data from one server while refusing to cooperate with others, causing those servers to lose their collateral despite the client having successfully retrieved her data. As we currently see no viable cryptographic solution to fully prevent this behavior, we instead discuss it from a rational perspective.

A straightforward mitigation is to introduce bilateral collateral from both clients and servers. For example, a fraction of the client's payment (*i.e.*, acting as client-side collateral) is only refunded to the client if a finalization transaction is confirmed on the ledger. Otherwise, the full payment is transferred to the server. In parallel, the server is required to over-collateralize to discourage server-side abandonment. Since forfeited server-side collateral is burned rather than transferred to the client, and the client risks losing the entire payment if she refuses to send a valid partial transaction, the rational strategy for the client is always to cooperate, even after retrieving the data.

Scalability optimization. The default design of our preservation phase requires servers to publish a test transaction for each iteration during the contract period. While this provides fine-grained auditability, it incurs a linear overhead in both blockchain storage and transaction fees.

Our optimization employs recursive zkSNARKs [6,8] over iterative proof generation, allowing a server to submit only several aggregated *succinct* proofs (and thus that many test transactions) as checkpoints. Beyond efficiency, this also offers a security benefit: reducing the number of recorded proofs on-chain limits the information that an adversary can exploit to mount adaptive replication soundness attacks (as discussed in Section 1.1). Moreover, these zkSNARKs, even used recursively, have been shown to be non-malleable [1, 17], further strengthening our security guarantees.

Retrieval policy and configurations. In the retrieval phase, we consider two configuration options: (1) a server may run $\mathsf{Extract}$ on her local replica and send a concealed version of the obtained data instead of a concealed replica. Accordingly, the adaptive client-fairness definition can be adjusted to accommodate this variant with only minor modifications; and (2) in deployments where multiple replicas of identical data are delegated to a server, she may perform *batch retrieval* by preparing a base finalization transaction $\tilde{\mathsf{tx}}_{fin}$ (see Section 2.1) that references as inputs all corresponding test transaction outputs, regardless of whether they have reached the contract period. This also partially mitigates the previously discussed risk of a client refusing to initiate retrievals from other replicas after obtaining the data from one.

Moreover, we discuss the early, on-demand data retrieval here. To enable it, we can add dedicated early-retrieval outputs to the contract transaction tx_{con}. The client first consumes such an output with an additional payment transaction, which must subsequently be consumed by a valid retrieval transaction (*i.e.*, same as the final retrieval transaction). This preserves our fairness guarantees for the *first* retrieval session. However, once a rational client learns the data, it may abort subsequent partial-transaction adaptations, causing the server to unilaterally lose collateral. Redirecting payments *unconditionally* to the server for deterring client-side abort would instead incentivize malicious servers to withhold partial transactions. This reflects the classical optimistic fair exchange limitation in the absence of a trusted third party and explains why deployed DSN protocols (*e.g.*, Filecoin [33]) adopt streaming, micropayment-based retrieval via Layer-2 channels to bound per-chunk risk rather than enforcing a single atomic exchange. We argue that our adaptor signature-based data retrieval mechanism can be made streaming-style by chunking the replica and executing on each chunk. However, our theoretical model deliberately follows the theoretically well-established fair exchange framework [41] instead of re-deriving a micropayment scheme. We acknowledge that applying the current approach to a streaming setting increases on-chain transactions and reduces scalability, but we regard this as an engineering trade-off orthogonal to our contribution on formalizing fair data retrieval for DSN.

C.2 Additional Security Considerations

We briefly discuss two additional security aspects of our DSN protocol. First, as mentioned in Section 3.3, we restate the *non-adaptive* client-fairness definition,

as originally proposed in [41], within our DSN syntax. Second, we address a potential privacy concern in the retrieval phase: our current design exposes the witness key w in plaintext when $\mathcal{S}$ completes tx_{fin} to enable $\mathcal{C}$'s data retrieval.

Client-fairness. The oracles are defined identically to those in the adaptive client-fairness setting. The experiment is provided in Figure 15.

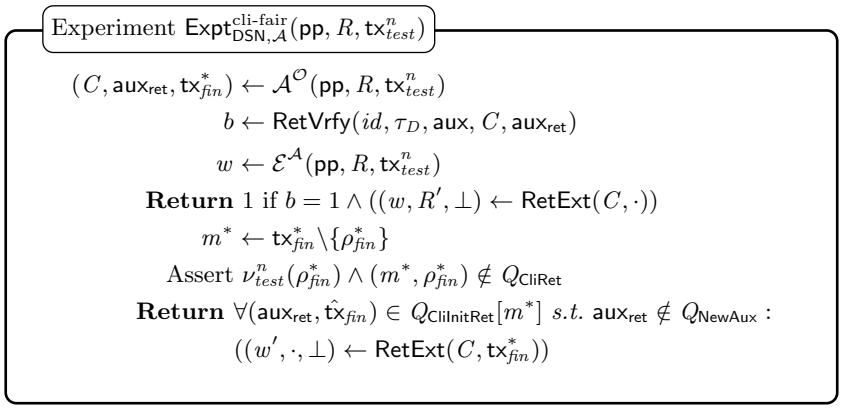


Fig. 15: The experiment for client-fairness. The adversary $\mathcal{A}$ receives $(\mathsf{pp}, R, \mathsf{tx}^n_{test})$ and is granted access to oracles $\mathcal{O} := \{\mathcal{O}_{\mathsf{CliRet}}, \mathcal{O}_{\mathsf{CliInitRet}}, \mathcal{O}_{\mathsf{NewAux}}\}$. Note that $\mathcal{O}_{\mathsf{CliDelPh}}$ is not necessary, as $\mathcal{A}$ receives valid $\mathsf{tx}^n_{test} \in \mathcal{L}$; and $\mathcal{O}_{\mathsf{SimAux}}$ is not provided given the non-adaptive setting. The winning condition of $\mathcal{A}$ is identical to that of adaptive client-fairness, except that it omits the freshness check against Q_{SimAux}.

Definition 28 (Client-Fairness). *A blockchain-aided DSN protocol has client-fairness if, for any PPT adversary $\mathcal{A}$ with access to oracles $\mathcal{O}_{\mathsf{CliRet}}, \mathcal{O}_{\mathsf{CliInitRet}}$, $\mathcal{O}_{\mathsf{NewAux}}$, there exists an extractor $\mathcal{E}^{\mathcal{A}}$, s.t. the following probability*

$$\Pr\left[\mathsf{Expt}^{\text{cli-fair}}_{\mathsf{DSN},\mathcal{A},\mathcal{E}}(\mathsf{pp}, R, \mathsf{tx}^n_{test}) = 1\right]$$

is $\mathsf{negl}(\lambda)$ *for any* $\lambda > 0, \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \cdot)$*, and any R and its corresponding* $\mathsf{tx}^n_{test} \in \mathcal{L}$.

Privacy concerns over publicly exposed witness key. One possible solution is to adopt an unlinkable adaptor signature, *e.g.*, the one proposed in [13, Generic Construction 2], which leverages the re-randomizability of instance-witness pairs in the hard relation underlying the scheme. Alternatively, we can employ a designated-receiver (client) approach: the witness is encrypted under the client's public key using a secure public-key encryption scheme, and the relation $\mathcal{R}_{\mathsf{InitRet}}$ is modified to additionally validate this encryption. We leave the integration and evaluation of these privacy-preserving extensions to future work.

C.3 Transaction Transition Illustration

The protocol state transition represented by the transaction inputs and outputs are illustrated in Figure 16, 17, and 18.

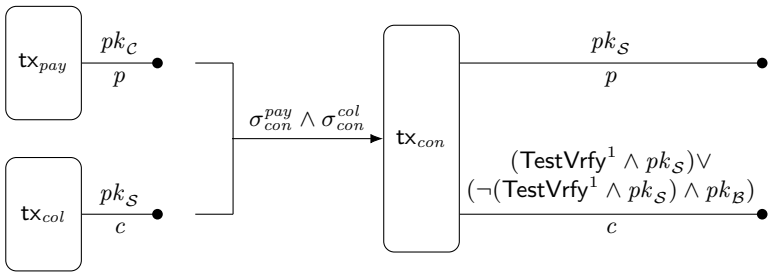


Fig. 16: State transition in the delegation phase. The payment and collateral outputs possessed by $\mathcal{C}$ and $\mathcal{S}$ are locked into a contract transaction tx_{con}. Its output commits to the first PoRep verification that initiates the preservation phase.

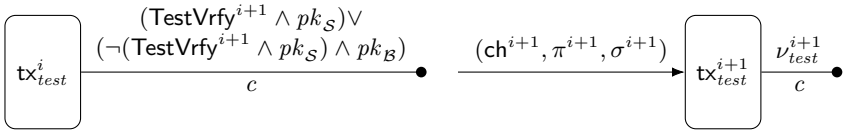


Fig. 17: State transition in the preservation phase. For all $i \in [n-1]$, each test transaction tx^{i}_{test} locks the collateral under a condition tied to a PoRep-based proof and signature produced solely by the server. A valid tuple $(\mathsf{ch}^{i+1}, \pi^{i+1}, \sigma^{i+1})$ advances the contract to the next test iteration.

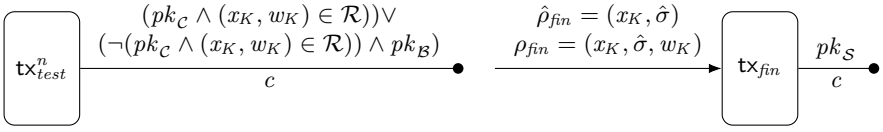


Fig. 18: State transition in the final data retrieval phase. The collateral locked in tx^{n}_{test} is consumed by the finalization transaction tx_{fin} only when a valid signature from $\mathcal{C}$ is provided, together with a valid instance-witness key pair $(x_K, w_K) \in \mathcal{R}$. This mechanism inherently resembles an adaptor signature scheme. Once confirmed on $\mathcal{L}$, tx_{fin} marks the end of the DSN protocol and enables $\mathcal{S}$ to reclaim her locked collateral.

D Scalability and Concrete Costs

One test transaction is submitted per test iteration. Although we do not conduct system-level benchmarks, concrete performance figures are available for candidate proof systems.

The on-chain storage overhead arises from the proof itself and the verification circuit embedded in the validator (cf. Remark 4). For Groth16 [29] on BLS12-381, supported in Plutus via CIP-0381 [39], a proof is 192 byte (B). For universal zkSNARKs such as PLONK [22], also particularly analyzed in [39], proof size is independent of the circuit size and remains at the kilobyte (KB) scale. As for the verification circuit, a Plutus PLONK verifier script occupies 6274 B, well within Cardano's 16 KB transaction limit [43]. Verification time achieves millisecond range on commodity CPUs [39].

These are Layer-1 numbers: the cost scales only with the number of test iterations. Nothing in our construction precludes porting it to an EUTxO Layer-2 protocol (Hydra) to improve scalability. However, we deliberately exclude such discussions from this work, as Layer-2 designs are beyond our current scope.

Finally, our protocol naturally supports multi-server and multi-client settings. PoRep is generic enough to allow identity- and session-specific replications (modeled via our metadata md and realizable, for example, via designated-target encryption). Each $(\mathcal{C}, \mathcal{S}, D, \mathsf{md})$-tuple therefore defines a unique replica, which is already accommodated in our current protocol description.